TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI

BỘ MÔN TOÁN HỌC ỨNG DỤNG



GIÁO TRÌNH
CÁC MÔ HÌNH TÍNH TOÁN

Giảng viên: Nguyễn Văn Hưng

Sinh viên: ……***MSSV***: ……***Lớp***: …….

Hà nội 01-2026

CÁC YÊU CẦU ĐỐI VỚI MÔN HỌC

1. Sinh viên đi học đúng giờ. Sinh viên vào lớp sau khi giáo viên đã bắt đầu bài giảng sẽ **không được vào lớp**.
2. Sinh viên ghi bài và làm các ví dụ vào trong quyển in để tính điểm.
3. ***Lịch nộp ví dụ các buổi học và các bài tập sẽ theo lịch trên Microsoft Teams.***
4. ***Sinh viên cài Micorosft Teams trên điện thoại hoặc máy tính để lấy số liệu cho ví dụ, bài tập, bài kiểm tra, bài thi.***
5. ***Trong giờ học tuyệt đối không sử dụng điện thoại. Nếu sử dụng điện thoại sẽ tính là sử dụng tài liệu.***
6. *Sẽ điểm danh đầu buổi học. Sinh viên vắng đầu buổi học sẽ tính vắng cả buổi.*
7. *Sinh viên nghỉ học chỉ khi có giấy phép của phòng Công tác sinh viên hoặc phòng Y tế.*
8. ***Sinh viên nghỉ 7 – 9 tiết và trung bình điểm 1 hoặc 2 dưới 4 sẽ không đủ điều kiện dự thi Nghỉ trên 12 tiết sẽ không được thi.***
9. Số liệu các ví dụ của buổi học sẽ được gửi trên Microsoft Teams trước mỗi buổi học 30 phút. Sinh viên hoàn thành ***điền các đáp số tương ứng vào các ô màu xanh lá cây và trình bày vào quyển ghi.*** Sau khi kết thúc buổi học thì sẽ đóng các ví dụ và chấm điểm.

Nếu sinh viên không tuân thủ theo các quy định nêu trên khi làm bài thì sẽ nhận điểm 0.

GIÁO TRÌNH MÔ HÌNH TOÁN

CÁCH TÍNH ĐIỂM

- Điểm 1 chiếm 25% tổng điểm, là điểm bài tập các buổi, điểm bài tập chương và vở ghi.
- Điểm 2 chiếm 25% tổng điểm: là điểm của các bài kiểm tra.
- Điểm cuối kỳ chiếm 50% tổng điểm.

TÀI LIỆU THAM KHẢO

1. ***Ngôn ngữ hình thức*** – Nguyễn Văn Ba. Nhà xuất bản khoa học Kỹ thuật.
2. ***Bài giảng môn học: Automata và ngôn ngữ hình thức*** – Hồ Văn Quân. Đại học Bách Khoa Hồ Chí Minh.
3. An Introduction to Formal Languages and Automata, FifthEdition – **Peter Linz**.

Các bài giảng của giáo sư, cho dù có đầy đủ, xúc tích đến đâu, có chứa chan tình yêu tri thức của bản thân giáo viên đến đâu, thì về thực chất, mà nói, đó chẳng qua cũng vẫn chỉ là chương trình, là những lời chỉ dẫn tuần tự để điều chỉnh trật tự nhận thức của sinh viên. Người nào chỉ biết ngồi nghe giáo sư giảng chứ bản thân mình trong lòng không cảm thấy khát khao học, thì có thể nói tất cả những điều người ấy nghe giảng ở trường đại học cũng sẽ chỉ như một tòa nhà xây trên cát mà thôi. **I.A. GONTCHAROV**

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

GIỚI THIỆU VỀ MÔN HỌC CÁC MÔ HÌNH TÍNH TOÁN (AUTOMATA VÀ NGÔN NGỮ HÌNH THỨC)

I. Mở đầu

Trong Khoa học máy tính, câu hỏi cốt lõi không chỉ là *máy tính làm được gì*, mà còn là *có những bài toán nào về nguyên tắc không thể giải được*, và *những bài toán nào có thể giải được với tài nguyên hữu hạn*. Để trả lời những câu hỏi này, các nhà khoa học đã xây dựng nên **các mô hình tính toán (models of computation)**, tiêu biểu là **automata** và **ngôn ngữ hình thức (formal languages)**. Đây là nền tảng lý thuyết quan trọng bậc nhất của Khoa học máy tính hiện đại.

II. Lịch sử ra đời và phát triển

1. Nguồn gốc từ toán học và logic

Cuối thế kỷ XIX, toán học đối mặt với cuộc khủng hoảng nền tảng khi xuất hiện các nghịch lý trong lý thuyết tập hợp (Russell, Cantor). Điều này thúc đẩy sự phát triển của:

- Logic hình thức (Frege, Russell, Hilbert).
- Hình thức hóa khái niệm suy luận và chứng minh.

David Hilbert đặt ra chương trình Hilbert với mục tiêu:

"*Mọi mệnh đề toán học đều có thể được quyết định bằng một thuật toán hữu hạn.*"

Câu hỏi này trực tiếp dẫn tới việc nghiên cứu khái niệm thuật toán và tính toán.

2. Giai đoạn khởi đầu (1930 – 1940): Những câu hỏi về tính toán.

Hilbert nêu ra ***Entscheidungsproblem***:

"*Có tồn tại một thuật toán tổng quát để quyết định mọi mệnh đề logic đúng hay sai hay không*?"

Để trả lời, cần trước hết định nghĩa chính xác thế nào là "*thuật toán*"**.**

Trong giai đoạn 1930 – 1936, ba mô hình tính toán quan trọng ra đời độc lập nhưng tương đương về sức mạnh:

- $1936 - Alan\ Turing$ và *máy Turing*: Đây là cột mốc quan trọng nhất. Turing đã đề xuất một mô hình máy lý thuyết có thể thực hiện bất kỳ thuật toán nào. Máy Turing chính là nền tảng cho kiến trúc máy tính hiện đại.
- $Alonzo\ Church$ và *phép tính Lambda* $(\lambda - calculus)$: Cùng thời điểm, Church đưa ra một mô hình tính toán dựa trên hàm số, sau này trở thành nền tảng cho các ngôn ngữ lập trình hàm như Lisp, Haskell.
- $Gödel, Church$: Hàm đệ quy.

Kết quả nổi tiếng trong giai đoạn này:

- *Church–Turing thesis* cho rằng mọi quá trình tính toán hiệu quả đều có thể được mô phỏng bởi máy Turing.
- Không tồn tại thuật toán tổng quát cho mọi bài toán.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Entscheidungsproblem có lời giải âm.

Giai đoạn này đã đặt nền tảng lý thuyết cho toàn bộ khoa học máy tính hiện đại và từ đây, *lý thuyết tính toán (Theory of Computation)* chính thức hình thành.

3. Giai đoạn hình thành (1940 – 1960): Thần kinh học và ngôn ngữ học

- 1943 – *McCulloch & Pitts*: Đề xuất mô hình logic của các tế bào thần kinh, đặt viên gạch đầu tiên cho Máy tự động hữu hạn (Finite Automata).
- 1956 – *Noam Chomsky vàphân cấp Chomsky*: Nhà ngôn ngữ học Chomsky đã phân loại các ngôn ngữ hình thức thành 4 loại (từ loại 3 đến loại 0). Đây là cầu nối quan trọng nhất kết hợp ngôn ngữ học với toán học và máy tính.
- *Stephen Kleene*: Giới thiệu khái niệm biểu thức chính quy (Regular Expressions) và chứng minh sự tương đương giữa chúng với máy tự động hữu hạn.

4. Giai đoạn hoàn thiện và mở rộng (1960 – nay)

- Các nhà khoa học như Stephen Cook và Richard Karp bắt đầu nghiên cứu về độ phức tạp tính toán (P vs NP), xác định xem máy tính có thể giải quyết các bài toán nhanh đến mức nào.
- Lý thuyết văn phạm phi ngữ cảnh được ứng dụng trực tiếp để thiết kế các trình biên dịch cho các ngôn ngữ như Algol, C.
- Phát triển automata ngăn xếp, automata tuyến tính, automata xác suất, automata song song.
- Mở rộng lý thuyết ngôn ngữ sang ngôn ngữ hình thức cho lập trình, đặc tả hệ thống, xác minh hình thức.

III. Ý nghĩa của các mô hình tính toán

1. Ý nghĩa lý thuyết

- Cung cấp định nghĩa chính xác cho khái niệm thuật toán và tính toán.
- Giúp phân loại các bài toán theo khả năng giải được và mức độ phức tạp.
- Là cơ sở cho các kết quả quan trọng như:

  + Bài toán không quyết định được.

  + Bài toán dừng.

  + Các lớp độ phức tạp (P, NP, PSPACE, …).

2. Ý nghĩa phương pháp luận

- Cho phép mô hình hóa các hệ thống phức tạp bằng các cấu trúc hình thức.
- Tạo ra ngôn ngữ chung để mô tả, phân tích và chứng minh tính đúng đắn của hệ thống.

IV. Ứng dụng của mô hình tính toán

1. Trong trình biên dịch và ngôn ngữ lập trình

Đây là ứng dụng thực tế nhất của mô hình tính toán:

- *Automata hữu hạn*: phân tích từ vựng (lexical analysis)
- *Văn phạm phi ngữ cảnh*: phân tích cú pháp (parsing)

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- *Ngôn ngữ hình thức*: định nghĩa cú pháp và ngữ nghĩa của ngôn ngữ lập trình.
- *Kiểm tra cú pháp và cấu trúc chương trình*.

2. Xác minh và kiểm chứng hệ thống

- Dùng máy tự động để chứng minh rằng các hệ thống quan trọng (như hệ thống điều khiển máy bay, phanh ABS) không bao giờ rơi vào trạng thái lỗi.
- Kiểm tra tính đúng đắn của : Phần mềm, giao thức mạng, hệ thống nhúng

3. Trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên

- Các mô hình văn phạm giúp máy tính hiểu cấu trúc câu của con người.
- Automata và grammar hỗ trợ: Phân tích câu, nhận dạng mẫu, xử lý chuỗi ký tự.

4. An toàn thông tin và hệ thống

- Phân tích luồng điều khiển.
- Phát hiện hành vi bất thường.
- Kiểm tra lỗ hổng logic trong hệ thống.
- Kiểm tra mã độc.

V. Các học phần liên quan

Nếu bạn theo đuổi ngành Khoa học máy tính, bạn sẽ gặp các khái niệm này trong các môn học sau:

- *Lý thuyết tính toán (Theory of Computation)*: Môn học cốt lõi nghiên cứu về Automata, Ngôn ngữ hình thức và Máy Turing.
- *Ngôn ngữ lập trình & Trình biên dịch (Compilers)*: Cách chuyển đổi mã nguồn thành mã máy.
- *Cấu trúc dữ liệu và Giải thuật*: Liên quan đến độ phức tạp tính toán và hiệu quả của các mô hình.
- *Thiết kế hệ thống số (Digital Logic Design)*: Ứng dụng máy tự động hữu hạn vào phần cứng.
- *Trí tuệ nhân tạo (AI)*: Các mô hình Markov, mạng thần kinh đời đầu có gốc rễ từ lý thuyết Automata.
- *An toàn thông tin*: Sử dụng lý thuyết ngôn ngữ để phát hiện mã độc hoặc tấn công mạng.

VI. Kết luận

Các mô hình tính toán, đặc biệt là automata và ngôn ngữ hình thức, không chỉ là những khái niệm trừu tượng mang tính lý thuyết mà còn là xương sống của khoa học máy tính hiện đại. Chúng kết nối toán học, logic và lập trình, giúp con người hiểu rõ giới hạn của tính toán, thiết kế hệ thống hiệu quả và đảm bảo tính đúng đắn cho các ứng dụng quan trọng.

Nếu Khoa học máy tính là một ngôi nhà, thì Automata và Ngôn ngữ hình thức chính là ***nền móng***. Bạn không cần nhìn thấy nó mỗi ngày khi ở trong nhà, nhưng không có nó, ngôi nhà không thể tồn tại.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

CHAPTER I. NGÔN NGỮ VÀ BIỂU DIỄN NGÔN NGỮ

§1. KHÁI NIỆM VỀ NGÔN NGỮ

I. Mở đầu

Ngôn ngữ là phương tiện để giao tiếp. Việc giao tiếp có thể là giữa người với người; giữa người với máy; giữa máy với máy. Tất cả chúng ta đều quen thuộc với ngôn ngữ tự nhiên. Đó là ngôn ngữ để con người giao tiếp với nhau, ví dụ như ngôn ngữ tiếng Việt, ngôn ngữ tiếng Anh, ngôn ngữ tiếp Pháp, … Từ điển định nghĩa ngôn ngữ một cách không chính thức như một hệ thống phù hợp để diễn đạt các ý tưởng, sự kiện hoặc khái niệm nhất định, bao gồm một tập hợp các ký hiệu và quy tắc để sử dụng chúng. Điều này cho chúng ta một ý tưởng trực quan về ngôn ngữ là gì, nhưng nó không đủ để làm định nghĩa cho việc nghiên cứu các ngôn ngữ chính thức. Đặc điểm của ngôn ngữ tự nhiên là quy tắc cú pháp thường rất phức tạp nhưng về mặt ngữ nghĩa thường thiếu chặt chẽ (ví dụ một câu có thể hiểu theo nhiều cách khác nhau tùy từng hoàn cảnh cụ thể).

Con người muốn giao tiếp với máy tính cũng phải thông qua ngôn ngữ. Tuy nhiên không thể sử dụng ngôn ngữ tự nhiên do tính thiếu chặt chẽ và đa nghĩa của ngôn ngữ tự nhiên. Ngôn ngữ này cần có cú pháp, ngữ nghĩa chặt chẽ, với một từ hay một câu thì nghĩa của nó là duy nhất không phụ thuộc ngữ cảnh. Ngôn ngữ như vậy được gọi là ngôn ngữ hình thức.

Con người muốn máy tính thực hiện công việc cần phải viết yêu cầu cho máy tính bằng ngôn ngữ mà máy có thể hiểu được. Việc viết yêu cầu cho máy là lập trình còn ngôn ngữ dùng để lập trình là ngôn ngữ lập trình. Ngôn ngữ lập trình cũng là ngôn ngữ hình thức.

Cả ngôn ngữ hình thức lẫn ngôn ngữ tự nhiên đều có thể xem như những tập các từ, tức là các xâu hữu hạn các phần tử của một bộ chữ cái cơ sở nào đó. Về mặt truyền thống, lý thuyết ngôn ngữ hình thức liên quan đến các đặc tả cú pháp của ngôn ngữ nhiều hơn là đến những vấn đề ngữ nghĩa. Một đặc tả về cú pháp của một ngôn ngữ có hữu hạn từ, ít nhất về nguyên tắc, có thể được cho bằng cách liệt kê các từ. Điều đó không thể áp dụng đối với các ngôn ngữ có vô hạn từ. Nhiệm vụ chính của lý thuyết ngôn ngữ hình thức là nghiên cứu các cách đặc tả hữu hạn của các ngôn ngữ vô hạn.

Lý thuyết tính toán cũng như của nhiều ngành khác nhau của nó, chẳng hạn mật mã học, có liên quan mật thiết với lý thuyết ngôn ngữ. Các tập vào và ra của một thiết bị tính toán có thể được xem như các ngôn ngữ và nói một cách sâu sắc hơn thì các mô hình tính toán có thể được đồng nhất với các lớp các đặc tả ngôn ngữ theo nghĩa mà trong bài giảng này chúng ta sẽ nêu chính xác hơn.

GIÁO TRÌNH MÔ HÌNH TOÁN

II. Một số khái niệm về ngôn ngữ

1. Bảng chữ và xâu

Định nghĩa 1.2.1.

Ví dụ 1.2.2.

- Bảng chữ tiếng Anh $\Sigma = \{a, b, c, \dots, z\}$.
- Bảng chữ tiếng Việt.
- Bảng chữ số thập phân $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
- Bảng chữ số nhị phân $\Sigma = \{0,1\}$.

Định nghĩa 1.2.3.

Quy ước.

- Các chữ cái của bảng chữ Σ được ký hiệu là $a, b, c, \dots$ (có thể thêm chỉ số).
- Các xâu được ký hiệu là $u,\ v,\ w, \dots$ (có thể thêm chỉ số).

Ví dụ 1.2.4.

- Từ "chinh" là một xâu trên bảng chữ tiếng Việt.
- Từ "rose" là một xâu trên bảng chữ tiếng Anh.
- Từ "010110" là một xâu trên bảng chữ nhị phân.
- Từ "2103147" là một xâu trên bảng chữ thập phân: $\Sigma = \{0,1,2,3,4,5,6,7,8,9\}$.

Định nghĩa 1.2.5.

Ví dụ 1.2.6.

- $|chinh| = 5$.
- $|rose| = 4$.
- $|010110| = 6$.
- $|2103147| = 7$.

Định nghĩa 1.2.7.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Nhận xét. $|\lambda| = 0$.

Định nghĩa 1.2.8.

Ví dụ 1.2.9. *Trong bảng chữ nhị phân* $\Sigma = \{0,1\}$*:*

-
-

2. Các phép toán trên xâu

Định nghĩa 1.2.10.

Ví dụ 1.2.11.

Định nghĩa 1.2.12.

Ví dụ 1.2.13.

-
-

Định nghĩa 1.2.14.

Ví dụ 1.2.15.

-

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

➢

Định nghĩa 1.2.14.

Ví dụ 1.2.17.

➢

➢

Định nghĩa 1.2.18.

Ví dụ 1.2.19. Xâu $u =$ "able was I ere I saw elba" có xâu đảo ngược là $u^R =$ "able was I ere I saw elba".

Đây là câu nói nổi tiếng của Napoleon Bonaparte khi bị lưu đày ra đảo Elba.

Nhận xét.

- ***Xâu rỗng λ là phần tử đơn vị đối với phép nhân ghép vì với mọi xâu u, ta có: $\lambda u = u\lambda = u$ nên xâu λ là tiền tố và hậu tố của mọi xâu u.***
- ***Với mọi xâu u, v ta có: $|uv| = |vu| = |u| + |v|$.***
- ***Với mọi xâu u và $n \in \mathbb{N}$ ta có $|u^n| = n|u|$.***
- ***uv và vu có thể là hai xâu khác nhau.***

III. Ngôn ngữ

1. Ngôn ngữ

Định nghĩa 1.3.1.

Ví dụ 1.3.2.

➢ *Ngôn ngữ tiếng Việt là ngôn ngữ trên bảng chữ cái tiếng Việt.*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- *Ngôn ngữ tiếng Anh là ngôn ngữ trên bảng chữ cái tiếng Anh.*
- $L = \{0,01,101,01001\}$ *là ngôn ngữ trên bảng chữ nhị phân.*
- $L = \Sigma^*$ *là ngôn ngữ trên bảng chữ* Σ.

Định nghĩa 1.3.3.

Chú ý: $\emptyset \neq \{\varepsilon\}$.

Định nghĩa 1.3.4.

Ví dụ 1.3.5.

- *Ngôn ngữ tiếng Việt là ngôn ngữ hữu hạn.*
- $\emptyset, \{\varepsilon\}$ *là các ngôn ngữ hữu hạn.*
- *Ngôn ngữ trên bảng chữ thập phân (tập các số tự nhiên) là ngôn ngữ vô hạn.*

2. Các phép toán trên ngôn ngữ

Định nghĩa 1.3.6. Cho hai ngôn ngữ L_1, L_2 ***cùng trên bảng chữ*** Σ.

- ***Hợp của hai ngôn ngữ*** L_1, L_2 ***trên bảng chữ*** Σ, ***ký hiệu là*** $L_1 \cup L_2$, ***là một ngôn ngữ trên bảng chữ*** Σ ***mà các từ thuộc vào một trong hai ngôn ngữ*** L_1, L_2.
- ***Giao của hai ngôn ngữ*** L_1, L_2 ***trên bảng chữ*** Σ, ***ký hiệu là*** $L_1 \cap L_2$, ***là một ngôn ngữ trên bảng chữ*** Σ ***mà các từ thuộc vào cả hai ngôn ngữ*** L_1, L_2.
- ***Phần bù của ngôn ngữ*** L_1 ***trên bảng chữ*** Σ, ***ký hiệu là*** $C_\Sigma(L)$, ***là một ngôn ngữ trên bảng chữ*** Σ, ***là tập hợp*** $\Sigma^* \backslash L_1$.

Ví dụ 1.3.7. Cho ngôn ngữ $L_1 = \{ab, ba, aba, baba\}$, $L_2 = \{ba, bab, bbaa, baba\}$ *trên bảng chữ* $\Sigma = \{a, b\}$. *Khi đó:*

$$L_1 \cup L_2 = \{ab, ba, aba, bab, baba, bbaa\}$$

$$L_1 \cap L_2 = \{ba, baba\}$$

Định nghĩa 1.3.8.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 1.3.9. *Cho ngôn ngữ* $L_1 = \{ab,\ ba\}$ *trên bảng chữ* $\Sigma_1 = \{a, b\}$, $L_2 = \{ac,\ cd\}$ *trên bảng chữ* $\Sigma_2 = \{a,\ c, d\}$. *Khi đó:*

Định nghĩa 1.3.10.

Ví dụ 1.3.11. *Cho ngôn ngữ* $L = \{aa, ab\}$ *trên bảng chữ* $\Sigma = \{a, b\}$. *Khi đó:*

Định nghĩa 1.3.12.

-
-

Nhận xét.

- $L^* = L^+ \cup \{\lambda\}$.
- ***Trong*** L^+ ***có thể chứa xâu*** λ ***nếu*** L ***chứa xâu*** λ.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 1.3.13. *Cho ngôn ngữ* $L = \{0,1\}$ *trên bảng chữ nhị phân* $\Sigma = \{0,1\}$. *Khi đó:*

- $L^0 = \{\lambda\}$.
- $L^1 = \{0,1\}$.
- $L^2 = \{00,01,10,11\}$
- $L^3 = \{000,001,010,011,100,101,110,111\}$.
- L^n *là tập các xâu nhị phân độ dài* $n (n \geq 0)$.
- L^* *là tập các xâu nhị phân,* L^+ *là tập các xâu nhị phận độ dài lớn hơn hoặc bằng* 1.

Ví dụ 1.3.14. *Cho bảng chữ* $\Sigma = \{a\}$. *Biểu diễn các ngôn ngữ sau:*

a) L_1: *tập các xâu trên* Σ *có độ dài lẻ.*

b) L_2: *tập các xâu trên* Σ *có độ dài chẵn ít nhất* 2.

Chú ý. Cho ba ngôn ngữ cùng trên bảng chữ $\Sigma = \{a, b, c\}$:

$$L_1 = \{a^n b^n c^n | n \geq 0\}$$

$$L_2 = \{a^n b^n | n \geq 0\}\{c^n | n \geq 0\} = \{a^n b^n | n \geq 0\}\{c^m | m \geq 0\} = \{a^n b^n | n \geq 0\}\{c\}^*$$

$$L_3 = \{a^n | n \geq 0\}\{b^n | n \geq 0\}\{c^n | n \geq 0\} = \{a^n | n \geq 0\}\{b^m | m \geq 0\}\{c^k | k \geq 0\}$$
$$= \{a\}^*\{b\}^*\{c\}^*$$

là ba ngôn ngữ khác nhau. Khi đó với ba xâu $u_1 = aaabbbccc, u_2 = aaabbbcc, u_3 = abbbcc$ ***thì:***

- ✓ ***Xâu*** u_1 ***thuộc vào cả ba ngôn ngữ*** L_1, L_2, L_3.
- ✓ ***Xâu*** u_2 ***thuộc vào ngôn ngữ*** L_2, L_3 ***nhưng không thuộc vào*** L_1.
- ✓ ***Xâu*** u_3 ***thuộc vào ngôn ngữ*** L_3 ***nhưng không thuộc vào*** L_1, L_2.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định nghĩa 1.3.15.

Ví dụ 1.3.16. *Ngôn ngữ* $L = \{star, desserts, live\}$ *trên bảng chữ tiếng Anh có ngôn ngữ ngược là:*

$$L^R = \{rats, stressed, evil\}$$

BÀI TẬP BUỔI 1

Bài tập xâu. *Cho xâu* $u =$ $v =$

Khi đó:

$|u| =$ $|v| =$

$uu =$

$uu^R =$

$u^R u =$

Xâu

Xâu

$uv =$

$vu =$

$vvv =$

Bài tập ngôn ngữ. *Cho ngôn ngữ* $L_1 = \{$ $\}$,

$L_2 = \{$ $\}$. *Khi đó:*

$L_1 L_1 =$

$L_1 L_1^R =$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

$L_1^R L_1 =$

$L_1 L_2 =$

$L_2 L_1 =$

§2. KHÁI NIỆM VĂN PHẠM VÀ AUTOMATA

I. Biểu diễn ngôn ngữ

Một ngôn ngữ L trên bảng chữ cái Σ là một tập con của tập Σ^*. Vấn đề đặt ra là đối với một ngôn ngữ L có cách nào chỉ rõ một xâu có thuộc L hay không.

Ví dụ 2.1.1. *Từ "bay" vừa là từ thuộc ngôn ngữ tiếng Việt, vừa thuộc ngôn ngữ tiếng Anh.*

Đối với các ngôn ngữ hữu hạn, để biểu diễn chúng thì chúng ta chỉ cần liệt kê ra tất cả các xâu thuộc vào ngôn ngữ.

Ví dụ 2.1.2. *Ngôn ngữ tiếng Việt là hữu hạn nên có thể liệt kê tất cả các từ tiếng Việt.*

Đối với ngôn ngữ vô hạn không phức tạp, chúng ta có thể xác định các xâu bằng cách chỉ ra đặc điểm cốt yếu của mỗi xâu.

Ví dụ 2.1.3.

-
-
-

Ví dụ 2.1.4. *L là tập các xâu uv sao cho u là xâu gồm toàn ký tự a có số chẵn ký tự a, v là xâu gồm toàn ký tự b và có độ dài ít nhất 2.*

Để nghiên cứu ngôn ngữ bằng toán học, chúng ta cần một phương pháp để mô tả chúng. Ngôn ngữ tự nhiên thường không chính xác và mơ hồ, vì vậy các mô tả ngôn ngữ thường không đủ. Ký hiệu tập hợp được sử dụng trong ví dụ 3.1.3 và 3.1.4 tuy phù hợp hơn, nhưng bị hạn chế. Để tiếp tục

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

chúng ta sẽ tìm hiểu một số cách định nghĩa ngôn ngữ mà nó hữu ích trong các trường hợp khác nhau. Ở đây chúng tôi sẽ giới thiệu một khái niệm phổ biến và mạnh mẽ, khái niệm về văn phạm.

II. Văn phạm

Văn phạm (theo ngôn ngữ học) là tập các quy tắc về cấu tạo từ và các quy tắc liên kết các từ thành câu. Văn phạm cho chúng ta biết liệu một câu cụ thể có được tạo ra đúng chuẩn hay không. Một quy tắc điển hình của ngữ pháp là "một câu bao gồm một chủ ngữ theo sau là một vị ngữ". Chúng ta có thể viết điều này một cách đơn giản và rõ ràng hơn như sau:

<Câu> ⟶ <Chủ ngữ> <Vị ngữ>

Tuy nhiên điều này là không đủ để mô tả các câu trong thực tế. Chúng ta sẽ định nghĩa cho các cấu trúc mới được giới thiệu "Chủ ngữ" và "Vị ngữ". Khi đó chúng ta sẽ thực hiện như sau:

<Chủ ngữ>⟶<Danh từ>

<Vị ngữ>⟶<Động từ>

và nếu chúng ta kết hợp các từ "Cây" và "Trời" với "mưa" và "ngủ", khi đó theo ngữ pháp chúng ta biết được các câu "Cây mưa" và "Trời ngủ" được tạo thành đúng chuẩn. Nếu chúng ta có thể đưa ra một văn phạm hoàn chỉnh thì về mặt lý thuyết, mọi câu thích hợp đều có thể được giải thích theo cách này.

Định nghĩa 2.2.1

-

-

Định nghĩa 2.2.2.

-

-

-

-

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Các quy tắc sản xuất là thành phần cốt lõi của văn phạm. Các sản xuất xác định cách văn phạm biến đổi một xâu này thành một xâu khác và thông qua đó, chúng sẽ xác định một ngôn ngữ được sinh bởi văn phạm.

Định nghĩa 2.2.3.

-

-

Chú ý. Dấu * thể hiện số bước suy dẫn không xác định (có thể là 0) để có thể nhận được xâu y từ xâu x.

- ✓ ***Nếu trong tập sản xuất P có một loạt các sản xuất có cùng vế trái***

$$\alpha \longrightarrow \beta_1, \alpha \longrightarrow \beta_2, \ldots, \alpha \longrightarrow \beta_n$$

thì các sản xuất này thường được viết gộp lại:

$$\alpha \longrightarrow \beta_1 | \beta_2 | \ldots | \beta_n$$

Nhận xét. $\Longrightarrow$ là quan hệ hai ngôi trên Σ^* còn $\Longrightarrow^*$ là bao đóng bắc cầu, phản xạ trên Σ^*.

Bằng cách áp dụng các quy tắc sản xuất theo các cách khác nhau thì một văn phạm có thể tạo ra nhiều xâu. Tập hợp tất cả các xâu gồm các ký hiệu kết thúc như vậy là ngôn ngữ được định nghĩa hay sinh bởi văn phạm.

Định nghĩa 2.2.4.

-

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

-

Quy ước.

- ◊ Các ký tự in hoa $A, B, C, \ldots, S$ (có thể thêm chỉ số) để ký hiệu các ký tự không kết thúc.
- ◊ Các ký tự thường $a, b, c, \ldots,$ (có thể thêm chỉ số) để ký hiệu các ký tự kết thúc.
- ◊ Các ký tự in hoa X, Y, Z (có thể thêm chỉ số) để ký hiệu các ký tự kết thúc và không kết thúc.
- ◊ Các ký tự thường u, v, x, y, z (có thể thêm chỉ số) biểu thị các xâu gồm toàn ký hiệu kết thúc.
- ◊ Các ký tự Hy Lạp $\alpha, \beta, \gamma, \ldots$ (có thể thêm chỉ số) biểu thị các xâu bất kỳ.

Ví dụ 2.2.5. *Cho văn phạm* $G = (\{S, A\}, \{a, b\}, S, \{S \longrightarrow aA, A \longrightarrow bS, A \longrightarrow a\})$.

a) Viết một dẫn xuất và xâu thuộc dẫn xuất.

b) Chỉ ra xâu thuộc $L(G)$ *và xâu dạng chuẩn.*

- Xâu

- Xâu

c) Tìm $L(G)$

Văn phạm G có thể xác định đầy đủ $L(G)$ nhưng không dễ dàng có được sự mô tả một cách chính xác về ngôn ngữ sinh bởi G. Tuy nhiên trong ví dụ này, kết quả là khá rõ ràng.

$$L(G) = \{(ab)^n a^2 | n \geq 0\} = \{ab\}^*\{a^2\}$$

Chúng ta có thể chứng minh kết quả này một cách dễ dàng. Nếu chúng ta nhận thấy hai quy tắc $S \longrightarrow aA, A \longrightarrow bS$ là đệ quy thì chúng ta có thể chứng minh bằng quy nạp toán học. Chúng ta sẽ chỉ ra rằng trong dãy dẫn xuất: $S \Longrightarrow u_1 \Longrightarrow u_2 \Longrightarrow u_3 \Longrightarrow u_4 \Longrightarrow \cdots \Longrightarrow u_{2n-1} \Longrightarrow u_{2n}$.

thì các xâu $u_{2i+1} = (ab)^i aA, u_{2i+2} = (ab)^{i+1} S$ với $i = \overline{0,n}$ và $n \in \mathbb{N}$.

Bước cơ sở. Với $i = 0$ thì

- $S = \lambda S = (ab)^0 S$ nên $u_0 = (ab)^0 S$.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- $S \Longrightarrow aA = \lambda aA = (ab)^0 aA$ nên $u_1 = (ab)^0 aA$.
- $S \Longrightarrow aA \Longrightarrow abS = (ab)^1 S$ nên $u_2 = (ab)^1 S$.

Bước quy nạp. Giả sử đẳng thức đúng đến $k = i\ (0 \leq i < n)$, tức là $u_{2i+1} = (ab)^i aA, u_{2i+2} = (ab)^{i+1} S$, ta cần chứng minh đẳng thức đúng với $k = i + 1$.

$$u_{2i+2} = (ab)^{i+1} S \Longrightarrow (ab)^{i+1} aA \Longrightarrow (ab)^{i+1} abS = (ab)^{i+2} S$$

nên $u_{2i+3} = (ab)^{i+1} aA$ hay $u_{2(i+1)+1} = (ab)^{i+1} aA$, $u_{2i+4} = (ab)^{i+2} S$ hay $u_{2(i+2)} = (ab)^{i+2} S$.

Vậy đẳng thức đúng với $k = i + 1$ nên các xâu $u_{2i+1} = (ab)^i aA, u_{2i+2} = (ab)^{i+1} S$ với $i = \overline{0, n}$. Từ đó suy ra $S \Longrightarrow u_{2n} = (ab)^n S \Longrightarrow (ab)^n aA \Longrightarrow (ab)^n aa = (ab)^n a^2$.

Để đơn giản ta có thể viết như sau:

S

hay S

d) Từ văn phạm G tìm văn phạm sinh ra ngôn ngữ $L_1 = \{a(ab)^n a^2 | n \geq 0\}$.

Ý tưởng là ta sẽ phát triển G bằng cách tạo thêm ký hiệu a ở tiền tố của các xâu $L(G)$. Việc này có thể thực hiện được bằng cách thực hiện với sản xuất $S \longrightarrow aA$ và những sản xuất khác có thể được chọn để A có thể suy ra ngôn ngữ trong đề bài. Khi đó ta có văn phạm:

Để

e) Chứng minh văn phạm $G_2 = (\{S_2, A_2, B_2\}, \{a, b\}, S_2, \{S_2 \longrightarrow A_2 a, A_2 \longrightarrow B_2 a | a, B_2 \longrightarrow A_2 b\})$ tương đương với văn phạm G.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

mà

Vậy

f) G là văn phạm thuộc nhóm nào?

Các

***Ví dụ 2.2.6**. Cho ngôn ngữ $L = \{01^n0^{m+2}1^2 | n, m \geq 0\}$.*

a) Tìm văn phạm G sinh ra L.

Xâu

vì

b) Tìm văn phạm G_1 tương đương với văn phạm G.

Xâu

vì

Vậy

c) G là văn phạm thuộc nhóm nào?

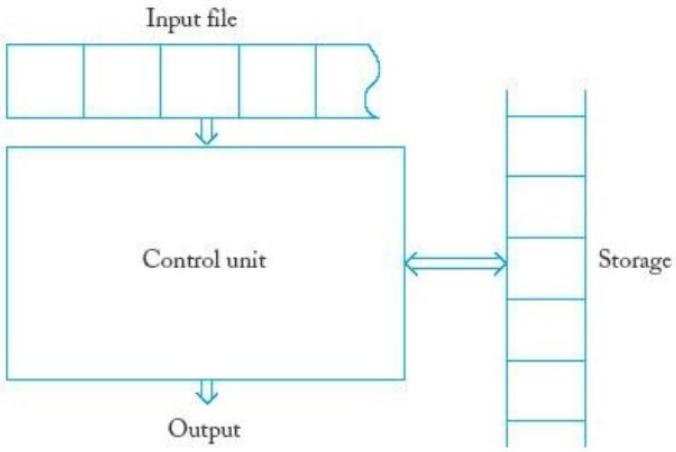
Các

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định nghĩa 2.2.7.

III. Automata

Một automata (máy tự động) là một mô hình trừu tượng của máy tính, vì vậy, mọi automata đều bao gồm một số tính năng cần thiết. Nó có một thiết bị để đọc đầu vào với giả thiết rằng tập đầu vào là một xâu trên bảng chữ nhất định và được viết trên một băng vào nhưng automata chỉ có thể đọc và không thể thay đổi. Xâu đầu vào được chia vào các ô, mỗi ô chứa một ký hiệu. Automata đọc xâu vào từ trái sang phải, mỗi lần một ký hiệu và có cách thức để phát hiện điểm kết thúc của mỗi xâu vào (bằng cách cảm nhận điều kiện kết thúc của xâu vào). Automata cũng có thể tạo ra đầu ra ở một vài dạng. Automata có thể có một thiết bị lưu trữ tạm thời, bao gồm một số ô nhớ không giới hạn, mỗi ô chứa một ký hiệu từ một bảng chữ (có thể khác với bảng chữ ở xâu vào). Automata có thể đọc và thay đổi nội dung trên các ô nhớ. Cuối cùng, automata có một bộ điều khiển mà ở mỗi thời điểm có thể thuộc vào một trong hữu hạn các trạng thái và nó có thể thay đổi các trạng thái theo một cách xác định. Hình dưới dây biểu diễn một sơ đồ một automata tổng quát:



Một automata được giả thiết hoạt động trong các khoảng thời gian biệt lập. Tại mỗi thời điểm, automata ở một trạng thái nhất định, và đầu đọc đọc một ký hiệu trên xâu vào. Trạng thái tiếp theo của bộ điều khiển được xác định bởi hàm chuyển trạng thái. Hàm chuyển trạng thái xác định trạng thái tiếp theo của bộ điều khiển dựa vào ký hiệu đọc được trên xâu vào, trạng thái hiện tại của bộ điều khiển và thông tin đang được lưu trữ trên bộ nhớ.

Trong quá trình chuyển trạng thái, bộ điều khiển có thể tạo ra đầu ra hoặc thay đổi thông tin trên bộ nhớ. Thuật ngữ "cấu hình" sẽ được sử dụng để chỉ trạng thái tức thời của bộ điều khiển, xâu vào và bộ nhớ hiện thời. Sự chuyển đổi của automata từ cấu hình này sang cấu hình khác gọi là một "bước chuyển".

Mô hình chung này bao gồm tất cả các automata mà ta nghiên cứu trong học phần này. Bộ điều khiển gồm hữu hạn các trạng thái là phổ biến với mọi trường hợp nhưng sự khác biệt giữa các automata đến từ phương thức tạo ra đầu ra và đặc tính của bộ lưu trữ tạm thời. Như chúng ta sẽ học, đặc tính của bộ lưu trữ tạm thời sẽ chi phối sức mạnh của các automata khác nhau.

Trong các phần tiếp theo, ta cần phân biệt giữa automata đơn định và automata không đơn định. Một automata đơn định là tại mỗi "bước chuyển" xác định một cách duy nhất bởi cấu hình hiện

tại. Nếu chúng ta biết trạng thái hiện tại của automata, ký hiệu trên xâu vào đang được đọc và bộ nhớ hiện tại thì chúng ta có thể dự đoán chính xác cấu hình tiếp theo của automata. Trong một automata không đơn định, điều này là không phải như vậy. Tại mỗi thời điểm, automata không đơn định có thể có một số bước chuyển khả thi, vì vậy chúng ta chỉ có thể dự đoán một tập hợp các cấu hình tiếp theo có thể có của automata. Mối quan hệ giữa automata đơn định và automata không đơn định đóng vai trò quan trọng trong nghiên cứu của chúng ta.

Một automata có đầu ra ở mức giới hạn đơn giản là "có" hoặc "không" được gọi là một "accepter". Với một xâu vào, "accepter" sẽ đoán nhận xâu vào hoặc không đoán nhận xâu vào. Một automata tổng quát hơn, có khả năng tạo ra các ký hiệu đầu ra gọi là một "transducer".

§3. PHÂN LOẠI VĂN PHẠM THEO CHOMSKY

I. Văn phạm tổng quát

1. Định nghĩa và ví dụ

Định nghĩa 3.1.1. Văn phạm $G = (V, T, S, P)$ được gọi là văn phạm tổng quát hay văn phạm không hạn chế hay văn phạm nhóm 0 nếu không có bất kỳ một ràng buộc nào đối với tập các sản xuất P.

- ***Ngôn ngữ do văn phạm tổng quát sinh ra gọi là ngôn ngữ tổng quát.***

Ví dụ 3.1.2*. Trong hóa học tính toán hoặc sinh học phân tử (DNA computing): Các sợi DNA có thể kết hợp, tách ra và biến đổi dựa trên các enzyme. Các quy tắc biến đổi DNA chính là các quy tắc của văn phạm tổng quát.*

Ví dụ: Một chuỗi gen AGT khi gặp enzyme X sẽ biến thành TCA. Đây là quy tắc $\alpha \longrightarrow \beta$ không giới hạn độ dài, đặc trưng của loại 0.

Ví dụ 3.1.3*. (Hệ thống phân loại và đóng gói thông minh). Giả sử bạn có một dây chuyền gồm các sản phẩm chạy hỗn loạn, và bạn cần một hệ thống có thể:*

- *Đếm số lượng sản phẩm.*
- *Kiểm tra tính đối xứng (ví dụ: bộ linh kiện phải đi kèm nhau).*
- *Sắp xếp lại vị trí linh kiện ngay trên băng chuyền.*

Trong khi các văn phạm cấp thấp hơn chỉ có thể "đếm" hoặc "nhận diện" theo một chiều, văn phạm tổng quát cho phép bạn thực hiện quy tắc:

"*Nếu thấy A đứng trước B, hãy đổi chỗ chúng và biến B thành C*"

2. Ý nghĩa của văn phạm tổng quát

Ý nghĩa lớn nhất của văn phạm tổng quát nằm ở *sức mạnh biểu diễn*:

- *Tính toán **được** (Computability): Một ngôn ngữ **được** sinh ra bởi văn phạm tổng quát nếu và chỉ nếu nó **được** chấp nhận bởi một máy Turing. **Điều** này có nghĩa là bất kỳ*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

thuật toán nào con người có thể viết ra đều có thể biểu diễn dưới dạng một văn phạm tổng quát.

- *Giới hạn cuối cùng: Nó đại diện cho ranh giới của những gì có thể mô tả được bằng toán học và logic trong ngôn ngữ. Nếu một ngôn ngữ không thể biểu diễn bằng văn phạm tổng quát, nó được coi là "không thể tính toán" (uncomputable).*

3. Ứng dụng của văn phạm tổng quát

Văn phạm tổng quát rất khó để triển khai trong thực tế (vì bài toán kiểm tra xem một chuỗi có thuộc ngôn ngữ này hay không là *không thể quyết định được* - Undecidable), nó vẫn có các ứng dụng quan trọng:

- *Lý thuyết Ngôn ngữ lập trình*: Dùng để nghiên cứu giới hạn của các trình biên dịch (compilers). Nó giúp các nhà khoa học máy tính hiểu được những cấu trúc ngôn ngữ nào là quá phức tạp để máy tính có thể xử lý hiệu quả.
- *Trí tuệ nhân tạo (AI) cổ điển*: Trong các hệ thống dựa trên quy tắc (Rule-based systems), văn phạm tổng quát mô phỏng cách tri thức được biến đổi và suy diễn.
- Sinh học tính toán: Mô hình hóa các chuỗi DNA phức tạp và sự tái tổ hợp di truyền, nơi các đoạn mã có thể di chuyển, đảo ngược hoặc thay đổi dựa trên ngữ cảnh xung quanh.
- *Xử lý ngôn ngữ tự nhiên (NLP)*: Mặc dù ngôn ngữ tự nhiên (tiếng Việt, tiếng Anh) thường được coi là nằm giữa Loại 1 và Loại 2, văn phạm tổng quát cung cấp khung lý thuyết để giải thích các hiện tượng ngôn ngữ cực kỳ phức tạp mà các mô hình đơn giản không làm được.

Mặc dù nó "vạn năng", nhưng văn phạm tổng quát có một nhược điểm chí mạng: *Bài toán dừng* (*The Halting Problem*). Nếu viết một chương trình dựa hoàn toàn trên văn phạm tổng quát, không thể chắc chắn 100% bằng toán học rằng chương trình đó sẽ chạy xong hay sẽ bị treo (lặp vô hạn) trừ khi bạn chạy nó. Do đó, trong thực tế, các ngôn ngữ lập trình thường cố gắng giới hạn ở các cấp độ thấp hơn để đảm bảo tính ổn định.

II. Văn phạm cảm ngữ cảnh

1. Định nghĩa và ví dụ

Định nghĩa 3.2.1.

GIÁO TRÌNH MÔ HÌNH TOÁN

-

Ví dụ 3.2.2. *Cho văn phạm* $G = (\{S, A, B\}, \{0, 1, 2\}, S, P)$ *với tập sản xuất:*

$$P = \{S \to 0SAB | 01B, BA \to AB, A \to 1, B \to 2\}$$

a) *Chứng minh văn phạm* G *là văn phạm cảm ngữ cảnh.*

Do

b) *Tìm một xâu thuộc* $L(G)$.

Vậy

2. Ý nghĩa của văn phạm cảm ngữ cảnh

- *Sức mạnh trung gian: Nó mạnh hơn văn phạm phi ngữ cảnh (dùng trong lập trình) nhưng yếu hơn văn phạm tổng quát (máy Turing). Nó có thể mô tả hầu hết các cấu trúc toán học và ngôn ngữ tự nhiên phức tạp.*
- *Tính quyết định (Decidability): Khác với văn phạm tổng quát (Loại 0), văn phạm cảm ngữ cảnh có tính quyết định. Nghĩa là luôn có một thuật toán để kiểm tra xem một chuỗi cho trước có thuộc ngôn ngữ đó hay không (dù tốn thời gian).*
- *Mô hình máy tương ứng: Ngôn ngữ cảm ngữ cảnh được chấp nhận bởi máy Turing giới hạn biên (Linear Bounded Automaton - LBA). Đây là một máy Turing mà bộ nhớ (băng ghi) bị giới hạn bởi độ dài của chuỗi đầu vào.*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

3. Ứng dụng thực tế

Dù ít được nhắc đến hơn văn phạm chính quy hay phi ngữ cảnh trong lập trình hằng ngày, văn phạm cảm ngữ cảnh lại cực kỳ quan trọng trong các lĩnh vực chuyên sâu:

- Ngôn ngữ tự nhiên (như tiếng Việt, tiếng Anh) thực chất là "cảm ngữ cảnh".

Ví dụ 3.2.3. *Trong tiếng Việt, từ "đường" có thể là "con đường" hoặc "đường ăn". Để hiểu nghĩa của nó, máy tính cần nhìn vào các từ xung quanh (ngữ cảnh). Các mô hình ngôn ngữ hiện đại (như Transformer/GPT) thực chất là các bộ máy xử lý ngữ cảnh cực kỳ phức tạp để giải quyết vấn đề này.*

- Kiểm tra kiểu trong lập trình (Type Checking): Hầu hết các ngôn ngữ lập trình (C++, Java, Python) không hoàn toàn là "phi ngữ cảnh".

Ví dụ 3.2.4. *Bạn không thể sử dụng biến x nếu chưa khai báo nó trước đó. Việc kiểm tra xem "biến đã khai báo chưa" là một bài toán cảm ngữ cảnh. Trình biên dịch phải nhìn lại phần "ngữ cảnh" phía trên của mã nguồn để xác định tính hợp lệ của dòng hiện tại.*

- Sinh học phân tử (Bioinformatics): Cấu trúc bậc hai của RNA hoặc các tương tác giữa các protein thường có dạng cảm ngữ cảnh. Các đoạn mã di truyền không hoạt động độc lập; chúng thay đổi chức năng dựa trên các đoạn mã nằm liền kề hoặc cách một khoảng xác định (long-range dependencies).
- Trong trình biên dịch (Compilers): Vì việc xử lý trực tiếp bằng văn phạm cảm ngữ cảnh (Loại 1) rất tốn kém về tài nguyên tính toán (độ phức tạp cực lớn), các kỹ sư máy tính thường dùng mẹo:

- Sử dụng văn phạm phi ngữ cảnh (CFG) để kiểm tra cấu trúc lệnh (ngoặc đơn, dấu chấm phẩy...).
- Sử dụng bảng ký hiệu (Symbol Table) để xử lý phần "cảm ngữ cảnh". Bảng này lưu trữ mọi biến đã khai báo. Khi trình biên dịch thấy một biến, nó tra cứu trong bảng (đây chính là hành động kiểm tra ngữ cảnh).

Nếu không có tính chất "cảm ngữ cảnh", máy tính sẽ không bao giờ hiểu được mối quan hệ giữa các phần khác nhau trong một chương trình, và chúng ta sẽ không có các trình biên dịch thông minh như ngày nay.

III. Văn phạm phi ngữ cảnh

Văn phạm phi ngữ cảnh (Context – Free Grammar – CFG) là loại văn phạm quan trọng nhất và được ứng dụng rộng rãi nhất trong khoa học máy tính, đặc biệt là trong việc thiết kế các ngôn ngữ lập trình.

1. Định nghĩa và ví dụ

Định nghĩa 3.3.1.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

•

Ví dụ 3.3.2. *Cho văn phạm:* $G = (\{S, A, B\}, \{0,1,2\}, S, \{S \to S2|A2, A \to 0A1|01\})$

a) Chứng minh rằng G *là văn phạm phi ngữ cảnh.*

Vế trái

b) Tìm $L(G)$.

Vậy

Ví dụ 3.3.3. *Cho ngôn ngữ* $L = \{aba^{n+1}b^{n+2}a^{m+3}|n, m \geq 0\}$.

a) Tìm văn phạm G *sinh ra* L.

Xâu

vì

S

Với

b) G *là văn phạm thuộc nhóm nào?*

Tập

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

-

-

Vậy G

Ví dụ 3.3.4. *Cho văn phạm:* $G = (\{S\}, \{a, b\}, S, \{S \to aSa|bSb|aa|bb|\lambda\})$

a) Chứng minh rằng G *là văn phạm phi ngữ cảnh.*

Tập

-

-

Vậy G

b) Viết 7 xâu khác nhau độ dài lớn hơn 8 khác nhau sinh bởi G.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

c) Tìm $L(G)$.

Ta sẽ chứng minh $L(G) = L = \{uu^R | u \in \{a,b\}^*\}$.

Chiều thuận. Chứng minh $L(G) \subseteq L$, tức là mọi xâu $v \in L(G)$ thì $v \in L$, tức là $v = uu^R$ với $u \in \{a,b\}^*$. Ta sẽ chứng minh bằng quy nạp theo số lần dẫn xuất từ S.

Gọi n là số dẫn xuất từ S đến xâu kết thúc $v \in L(G)$.

Bước cơ sở. Với $n = 1$ ta có:

- $S \Longrightarrow \lambda = \lambda\lambda^R \in L(G)$ mà $\lambda\lambda^R \in L$.
- $S \Longrightarrow aa = aa^R \in L(G)$ mà $aa^R \in L$.
- $S \Longrightarrow bb = bb^R \in L(G)$ mà $bb^R \in L$.

Vậy bài toán đúng với số lần dẫn xuất là 1 từ S để đến được xâu kết thúc thuộc $L(G)$ và xâu này cũng thuộc L.

Bước quy nạp. Giả sử sau n ($n \geq 1$) lần suy dẫn ta có $S \Longrightarrow^* v$ với $v = uu^R$. Ta cần chứng minh sau $n+1$ lần suy dẫn cũng sẽ đến được xâu có dạng uu^R với $u \in \{a,b\}^*$.

Trường hợp 1. Lần suy dẫn đầu tiên ta sử dụng sản xuất $S \Longrightarrow aSa$. Khi đó từ S thuộc xâu aSa, sau n lần suy diễn ta được xâu $v = uu^R \in L(G)$ với $u \in \{a,b\}^*$. Khi đó

$$S \Longrightarrow aSa \Longrightarrow^* auu^Ra = auu^Ra^R = (au)(ua)^R$$

mà $au \in L(G)$ nên sau $n+1$ lần suy dẫn cũng sẽ đến được xâu có dạng uu^R với $u \in \{a,b\}^*$.

Trường hợp 2. Lần suy dẫn đầu tiên ta sử dụng sản xuất $S \Longrightarrow bSb$. Khi đó từ S thuộc xâu bSb, sau n lần suy diễn ta được xâu $v = uu^R \in L(G)$ với $u \in \{a,b\}^*$. Khi đó

$$S \Longrightarrow bSb \Longrightarrow^* buu^Rb = buu^Rb^R = (bu)(ub)^R$$

mà $au \in L(G)$ nên sau $n+1$ lần suy dẫn cũng sẽ đến được xâu có dạng uu^R với $u \in \{a,b\}^*$.

Theo nguyên lý quy nạp toán học, sau n lần suy diễn bất kỳ từ S, ta đến được các xâu v có dạng uu^R với $u \in \{a,b\}^*$, tức là $v = uu^R \in L$.

Vậy $L(G) \subseteq L$.

Chiều đảo. Ta sẽ chứng minh $L \subseteq L(G)$, tức là mọi xâu $v = uu^R \in L$ với $u \in \{a,b\}^*$ thì $uu^R \in L(G)$. Ta sẽ chứng minh bằng chiều dài của xâu uu^R.

Đặt $2n = |v| = |uu^R|$ với $u \in \{a,b\}^*$.

Bước cơ sở:

- Với $n = 0$ thì $v = \lambda = \lambda\lambda^R \in L$ mà $S \Longrightarrow \lambda$ nên $\lambda \in L(G)$
- Với $n = 1$ thì
  + Nếu $v = aa = aa^R \in L$ mà $S \Longrightarrow aa$ nên $aa \in L(G)$.
  + Nếu $v = bb = bb^R \in L$ mà $S \Longrightarrow bb$ nên $bb \in L(G)$.

Bước quy nạp. Giả sử xâu $v \in L$ với $|v| = 2n$ thì $v \in L(G)$. Ta sẽ chứng minh nó cũng đúng với các xâu $v \in L$ với $|v| = 2(n+1)$.

GIÁO TRÌNH MÔ HÌNH TOÁN

Do $v \in L$ và $|v| = 2(n+1)$ nên $v = av_1a$ hoặc $v = bv_1b$ với $v_1 \in L$ và $|v_1| = 2n$. Khi đó theo giả thiết quy nạp, $v_1 \in L(G)$ nên $S \Longrightarrow^* v_1$.

Trường hợp 1. $v = av_1a$ thì $S \Longrightarrow aSa \Longrightarrow^* av_1a = v$ nên $v \in L(G)$.

Trường hợp 2. $v = bv_1b$ thì $S \Longrightarrow bSb \Longrightarrow^* bv_1b = v$ nên $v \in L(G)$.

Theo nguyên lý quy nạp toán học, xâu $v \in L$ mà $|v| = 2(n+1)$ thì $v \in L(G)$ nên $L \subseteq L(G)$.

Vậy $L = L(G)$ hay G sinh ra ngôn ngữ $\{uu^R | u \in \{a,b\}^*\}$.

Ví dụ 3.3.5*. Xét xem đoạn chương trình dưới đây thuộc lớp văn phạm cảm ngữ cảnh hay phi ngữ cảnh:*

```
Program Test;
var x: integer;
    Begin
        x := 10; {Hợp lệ}
        y := 5;  {Lỗi: y chưa được khai báo}
    End;
```

- *Nếu là chương trình thuộc lớp văn phạm phi ngữ cảnh*: Trình biên dịch chỉ kiểm tra xem cấu trúc câu lệnh $y := 5$; có đúng dạng $biến := giá\ trị$; hay không. Nó không quan tâm y là ai, vì nó không có khả năng "nhìn lại" hoặc "nhìn xung quanh" để kiểm tra danh sách biến đã khai báo.
- *Nếu chương trình thuộc lớp văn phạm cảm ngữ cảnh*: Trình biên dịch buộc phải kiểm tra *ngữ cảnh* (phần khai báo var phía trên). Câu lệnh $y := 5$ chỉ được coi là "đúng ngữ pháp" *nếu* trong ngữ cảnh trước đó đã xuất hiện chuỗi $var\ y$.

2. Ý nghĩa của văn phạm phi ngữ cảnh

- *Cấu trúc phân cấp: CFG có khả năng mô tả các cấu trúc lồng nhau vô hạn. Điều này giúp nó vượt xa văn phạm chính quy.*
- *Mô hình máy tương ứng: Ngôn ngữ phi ngữ cảnh được chấp nhận bởi automata đẩy xuống (Pushdown Automata - PDA). Đây là một auotmata hữu hạn được trang bị thêm một ngăn xếp (Stack) để ghi nhớ dữ liệu.*
- *Tính hiệu quả: Các thuật toán phân tích cú pháp (parsing) cho CFG (như thuật toán CYK hoặc Earley) rất hiệu quả, cho phép máy tính xử lý các tệp mã nguồn khổng lồ trong thời gian ngắn.*

3. Ứng dụng của văn phạm phi ngữ cảnh

Văn phạm phi ngữ cảnh là "xương sống" của hầu hết các công nghệ xử lý ngôn ngữ hiện nay.

- Thiết kế trình biên dịch (Compilers): Hầu hết cú pháp của các ngôn ngữ lập trình như C, Java, Python được định nghĩa bằng CFG (sử dụng ký pháp Backus-Naur Form).

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 3.3.6. *Cấu trúc các cặp ngoặc lồng nhau:* $If\ ((a > b)\ \&\&\ (c < d))$

Trình biên dịch sử dụng Stack để đảm bảo mọi dấu ngoặc mở đều có dấu ngoặc đóng tương ứng.

- Trình phân tích cú pháp (Parser): Các công cụ nổi tiếng như **Yacc** hay **Bison** được dùng để tạo ra trình biên dịch cho các ngôn ngữ như C, C++, hay SQL thực chất là những cỗ máy thực thi các quy tắc của văn phạm phi ngữ cảnh.
- Phân tích cấu trúc dữ liệu (JSON, XML, HTML): Các định dạng dữ liệu hiện đại đều có cấu trúc cây (tree-based). Trong XML: $< tag > \ldots </tag >$. Thẻ đóng phải khớp với thẻ mở gần nhất. Đây chính là biểu hiện của một ngôn ngữ phi ngữ cảnh.
- Biểu thức toán học: Các máy tính bỏ túi xử lý các biểu thức như

$$(1 + 2) * \frac{3}{5 - 1}$$

dựa trên các quy tắc phi ngữ cảnh để xác định thứ tự ưu tiên của các phép toán.

- Trình soạn thảo văn bản (Code Editor): Các tính năng như *"Highlight matching brackets"* (tô sáng cặp ngoặc tương ứng) trong VS Code hay Sublime Text hoạt động dựa trên logic của văn phạm phi ngữ cảnh.
- Ngôn ngữ học : CFG được sử dụng để phân tích cấu trúc cú pháp của câu trong ngôn ngữ tự nhiên (S -> NP VP - Chủ ngữ + Vị ngữ). Mặc dù ngôn ngữ tự nhiên có những yếu tố "cảm ngữ cảnh", nhưng CFG vẫn là công cụ nền tảng để sơ đồ hóa câu văn.

IV. Văn phạm chính quy

Văn phạm chính quy (Regular Grammar) là cấp độ thấp nhất nhưng lại là cấp độ được sử dụng phổ biến nhất trong thực tế hằng ngày của một lập trình viên. Nó đại diện cho những ngôn ngữ có cấu trúc đơn giản, lặp đi lặp lại và không có tính chất lồng nhau phức tạp.

1. Văn phạm chính quy

Định nghĩa 3.4.1.

GIÁO TRÌNH MÔ HÌNH TOÁN

-

Chú ý. Trong văn phạm chính quy, có nhiều nhất một ký hiệu không kết thúc nằm ở vế phải của tất cả các sản xuất. Hơn nữa ký hiệu không kết thúc phải nhất quán hoặc là ký hiệu bên trái hoặc là ký hiệu bên phải của tất cả các sản xuất.

Ví dụ 3.4.2. *Cho văn phạm* $G = (\{S, A, B\}, \{0,1,2\}, S, \{S \to 0S|0A, A \to 1A|1B, B \to 2B|2\})$.

a) *Chứng minh văn phạm* G *là văn chính quy.*

Các

b) *Tìm* $L(G)$.

S

Vậy

Với

Ví dụ 3.4.3.

a) *Xây dựng văn phạm* G *trên bảng chữ nhị phân sinh ra các xâu có tiền tố* 00*, chứa xâu con* 101 *và có hậu tố là* 00.

G

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

$G =$

thỏa mãn

S

Với

b) Chứng minh G là văn phạm chính quy.

Các

c) Xây dựng FA M_1 tương đương với G.

Do

Do

Do

Do

Do

Do

Do

Do

Do

Vậy FA

d) Xây dựng DFA M_2 tương đương với G.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

δ'	0	1

Vậy

2. Văn phạm tuyến tính

Định lý 3.4.4. Nếu $G = (V, T, S, P)$ là văn phạm tuyến tính trái thì tồn tại văn phạm tuyến tính phải tương đương với G.

Xây dựng văn phạm tuyến tính phải $G' = (V, T, S, P')$ như sau: Nếu $A \longrightarrow Bu$ và $A \longrightarrow u$ $(A, B \in V, u \in T^*)$ là các sản xuất trong P thì thêm vào P' các sản xuất $A \longrightarrow u^R B$ và $A \longrightarrow u$. Khi đó có thể chứng minh được $L(G') = L(G)$.

Ví dụ 3.4.5. Văn phạm $G = (\{S, A, B\}, \{a, b\}, S, \{S \to abA|Bb, A \to Aba|aB, B \to a|\lambda\})$ không là văn phạm chính quy mặc dù các sản xuất của G hoặc có dạng tuyến tính trái, hoặc có dạng tuyến tính phải nhưng G không ở dạng tuyến tính trái cũng như tuyến tính phải.

Các

Chú ý. Một văn phạm tuyến tính là một văn phạm có nhiều nhất một ký hiệu không kết thúc nằm ở vế phải của tất cả các sản xuất và không hạn chế vị trí của biến này.

- ✓ ***Một văn phạm chính quy luôn là văn phạm tuyến tính nhưng văn phạm tuyến tính chưa chắc đã là văn phạm chính quy.***

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

3. Một số ngôn ngữ chính quy

Ví dụ 3.4.6. *Cho bảng chữ* $\Sigma = \{a_1, a_2, \ldots, a_n\}$. *Chứng minh các ngôn ngữ sau đều là các ngôn ngữ chính quy:*

a) $L_1 = \{a_1 a_2 \ldots a_n\}$.

Văn phạm

nên

b) $L_2 = \Sigma^*$

Văn phạm

nên

c) $L_3 = \Sigma^+$

Văn phạm

- Nếu
- Nếu

nên

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

d) $L_4 = \emptyset$

Văn phạm

nên

Ví dụ 3.4.6. *Một địa chỉ email thường có dạng* $Username@Domain.Extension$ *với* $Username, Domain, Extension$ *là các xâu gồm các chữ cái tiếng Anh (cả chữ hoa và chữ thường), chữ số và có thể thêm các dấu ".", ",", "_", các ký hiệu "%", "−", "+", "∗", "/", "=","!", "#", "$", "&", "′", "?", "^","{", "|", "}", "~".*

a) *Xây dựng văn phạm* G *sinh ra địa chỉ email. Chứng minh* G *là văn phạm chính quy.*

Văn phạm $G = (\{S, Username, Domain, Extension\}, \Sigma, S, P)$ với Σ là tập các ký tự trong địa chỉ email có thể có, P là tập sản xuất có dạng:

$$S \longrightarrow [\text{ký tự}]S|@Domain$$

$$Domain \longrightarrow [\text{ký tự}]Domain|.Extension$$

$$Extension \longrightarrow [\text{ký tự}]Extension|[\text{ký tự}]Extension_1$$

$$Extension_1 \longrightarrow [\text{ký tự}]$$

Khi đó một email là $hoa12@huce.edu.vn$ sẽ được văn phạm G sinh ra như sau:

$S \Longrightarrow hS \Longrightarrow hoS \Longrightarrow hoaS \Longrightarrow hoa1S \Longrightarrow hoa12S \Longrightarrow hoa12@Domain \Longrightarrow hoa12@hDomain \Longrightarrow hoa12@huDomain \Longrightarrow hoa12@hucDomain \Longrightarrow hoa12@huceDomain \Longrightarrow hoa12@huce.Extension \Longrightarrow hoa12@huce.eExtension \Longrightarrow hoa12@huce.edExtension \Longrightarrow hoa12@huce.eduExtension \Longrightarrow hoa12@huce.edu.Extension \Longrightarrow hoa12@huce.edu.vExtension_1 \Longrightarrow hoa12@huce.edu.vn$

Các sản xuất trong P đều có dạngtuyến tính phải nên G là văn phạm chính quy.

b) *Xây dựng FA* M *sinh ra địa chỉ email.*

FA $M = (\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}, \Sigma, q_0, \delta, \{q_6\})$ với Σ là tập các ký tự trong địa chỉ email có thể có với hàm chuyển trạng thái δ như sau:

$$\delta(q_0, [\text{ký tự}]) = \{q_1\}$$

$$\delta(q_1, [\text{ký tự}]) = \{q_1\}$$

$$\delta(q_1, @) = \{q_2\}$$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

$$\delta(q_2, [\text{ký tự}]) = \{q_3\}$$

$$\delta(q_3, [\text{ký tự}]) = \{q_3\}$$

$$\delta(q_3, .) = \{q_4\}$$

$$\delta(q_4, [\text{ký tự}]) = \{q_4, q_5\}$$

$$\delta(q_5, [\text{ký tự}]) = \{q_6\}$$

Khi đó một email là $hoa12@huce.edu.vn$ sẽ FA M đoán nhận như sau:

$q_0 hoa12@huce.edu.vn \Longrightarrow q_1 oa12@huce.edu.vn \Longrightarrow q_1 a12@huce.edu.vn \Longrightarrow q_1 12@huce.edu.vn \Longrightarrow q_1 2@huce.edu.vn \Longrightarrow q_1 @huce.edu.vn \Longrightarrow q_2 huce.edu.vn \Longrightarrow q_3 uce.edu.vn \Longrightarrow q_3 ce.edu.vn \Longrightarrow q_3 e.edu.vn \Longrightarrow q_3 .edu.vn \Longrightarrow q_4 edu.vn \Longrightarrow q_4 du.vn \Longrightarrow q_4 u.vn \Longrightarrow q_4 .vn \Longrightarrow q_4 vn \Longrightarrow q_5 n \Longrightarrow q_6 \in \{q_6\} = F$

Ứng dụng thực tế của mô hình này

Khi bạn hiểu được sơ đồ trạng thái, bạn sẽ biết cách tối ưu Regex:

- Hiệu năng: DFA chạy cực nhanh vì nó không bao giờ phải quay lui (backtracking) nếu bạn viết Regex tốt.
- Bảo mật: Tránh được lỗi ReDoS (Regular Expression Denial of Service) – lỗi xảy ra khi Regex quá phức tạp khiến máy tính tốn quá nhiều trạng thái trung gian để xử lý một chuỗi đầu vào độc hại.

4. Ý nghĩa của văn phạm chính quy

- *Tính hữu hạn: Nó tương đương với automata hữu hạn (Finite Automata - FA). Điều này có nghĩa là máy tính chỉ cần một lượng bộ nhớ cố định và cực kỳ ít để nhận diện ngôn ngữ này. Nó không cần ngăn xếp (stack) như văn phạm **cảm ngữ cảnh**.*
- *Tốc độ cực nhanh: Vì cấu trúc đơn giản, việc kiểm tra một chuỗi có chính quy hay không diễn ra gần như tức thời, với độ phức tạp thời gian là tuyến tính $O(n)$.*
- *Biểu thức chính quy (Regex): Đây là công cụ cụ thể hóa văn phạm chính quy vào lập trình thực tế.*

5. Ứng dụng của văn phạm chính quy

- Biểu thức chính quy (Regular Expressions - Regex): Đây là ứng dụng nổi tiếng nhất. Mọi ngôn ngữ lập trình (Python, JS, Java...) đều có thư viện Regex để:

- Kiểm tra định dạng email.
- Kiểm tra số điện thoại: Đảm bảo chuỗi chỉ chứa các chữ số và có độ dài đúng quy định.
- Tìm kiếm và thay thế: Tìm tất cả các từ "Color" để đổi thành "Colour" trong một văn bản lớn.
- Bộ phân tích từ vựng (Lexical Analyzer / Scanner): Trong trình biên dịch, trước khi kiểm tra cú pháp phức tạp, máy tính sẽ dùng văn phạm chính quy để "quét" mã

CÁC TRÌNH MÔ HÌNH TÍNH TOÁN

nguồn và phân loại các thành phần cơ bản (Tokens) như: Từ khóa (if, else, while); Tên biến (Identifier); Các toán tử (+, -, *, /).

- Thiết kế phần cứng (Digital Logic Design): Các mạch điều khiển trong đèn giao thông, máy bán nước tự động, hay các bộ vi xử lý đơn giản hoạt động dựa trên máy tự động hữu hạn — mô hình vật lý của văn phạm chính quy.
- Tìm kiếm trong văn bản (Grep, Sed, Awk): Các công cụ dòng lệnh trên Linux sử dụng văn phạm chính quy để lọc dữ liệu từ các file log khổng lồ của hệ thống.

V. So sánh các văn phạm

1. Bảng so sánh các văn phạm

Tiêu chí	*Chính quy*	*Phi ngữ cảnh*	*Cảm ngữ cảnh*	*Tổng quát*
Mô hình máy	Automata hữu hạn	Automata đẩy xuống	Automata giới hạn biên	Máy Turing
Bộ nhớ	Không có bộ nhớ phụ (chỉ có trạng thái hữu hạn)	Ngăn xếp (Stack) - LIFO (Vào sau ra trước)	Băng ghi giới hạn (Bằng độ dài chuỗi đầu vào)	Băng ghi vô hạn (Có thể đọc/ghi tùy ý)
Quy tắc thế	$A \longrightarrow aB$ hoặc $A \longrightarrow a$	$A \longrightarrow \alpha$	$\alpha_1 A \alpha_2 \longrightarrow \beta$	$\alpha \longrightarrow \beta \ (\lvert\alpha\rvert \geq 1)$
Cấu trúc đặc trưng	Chuỗi lặp, mẫu ký tự đơn giản.	Cấu trúc lồng nhau, đệ quy, đối xứng gương.	Cấu trúc phụ thuộc ngữ cảnh, sao chép chuỗi	Mọi cấu trúc có thể mô tả bằng thuật toán.
Khả năng	Nhận diện từ khóa, định dạng số, email	Phân tích cú pháp ngoặc	Kiểm tra khai báo biến, kiểm tra kiểu dữ liệu, ngôn ngữ tự nhiên.	Giải quyết mọi bài toán có thể tính toán được.
Giới hạn	Không thể đếm hoặc nhớ các cặp lồng nhau	Không thể khớp các cấu trúc song song	Độ phức tạp tính toán rất cao (NP-complete).	Bài toán dừng: Không thể chắc chắn máy sẽ dừng hay chạy mãi mãi.

2. Phân tích sâu các đặc điểm cốt lõi của văn phạm

a) Sự khác biệt về bộ nhớ (Cơ chế lưu trữ)

- *Văn phạm chính quy*: Giống như một người đi bộ không có giấy bút, chỉ nhớ mình đang ở ngã tư nào.
- *Văn phạm phi ngữ cảnh*: Có một chồng đĩa (Stack). Bạn có thể đặt thêm đĩa lên hoặc lấy đĩa trên cùng ra. Điều này giúp máy "nhớ" được mình đã mở bao nhiêu dấu ngoặc để đóng cho đúng.
- *Văn phạm cảm ngữ cảnh*: Có một đoạn giấy vừa đủ bằng độ dài bài toán. Bạn có thể viết đi viết lại trên đó để kiểm tra các mối liên hệ phức tạp.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- *Văn phạm tổng quát*: Có một cuộn giấy dài vô tận. Bạn có thể ghi chép, tính toán nháp và lưu trữ mọi thứ mà không sợ hết chỗ.

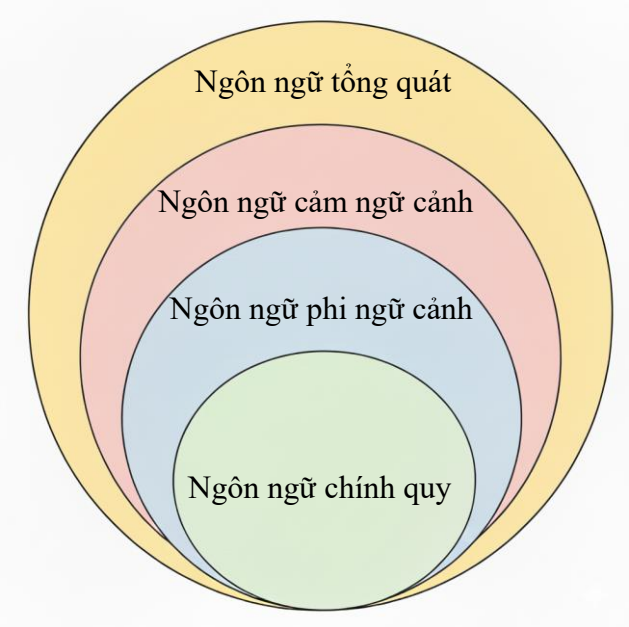
b) Cấu trúc đặc trưng và khả năng "đếm"

- *Văn phạm chính quy*: Chỉ có thể nhận diện các mẫu như "Một chuỗi các chữ số". Nó không thể so sánh số lượng giữa hai nhóm (ví dụ không thể biết số chữ số 0 và số chữ số 1 trong xâu có bằng nhau hay không).
- *Văn phạm phi ngữ cảnh*: Có thể đếm theo cặp (ví dụ số chữ số 0 bằng số chữ số 1 như trong xâu 0^n1^n).
- *Văn phạm cảm ngữ cảnh*: Có thể đếm bộ ba hoặc nhiều hơn (ví dụ $0^n1^n2^n$ hoặc kiểm tra xem một biến đã xuất hiện ở phần trên của chương trình chưa).

c) Các sản xuất

- Trong văn phạm phi ngữ cảnh, vế trái chỉ có một ký hiệu không kết thúc (A). Nó không quan tâm hàng xóm là ai
- Trong văn phạm cảm ngữ cảnh, vế trái có hàng xóm và phải chứa ký hiệu không kết thúc $(\alpha_1 A \alpha_2)$. Nó chỉ thay đổi nếu có đúng hàng xóm thân thiết bao quanh.
- Trong văn phạm tổng quát, vế trái là bất cứ thứ gì. Sự biến đổi diễn ra hoàn toàn tự do.

Nhận xét. Lớp các ngôn ngữ chính quy ⊂ Lớp các ngôn ngữ phi ngữ cảnh ⊂ Lớp các ngôn ngữ cảm ngữ cảnh ⊂ Lớp các ngôn ngữ tổng quát.



BÀI TẬP BUỔI 2

Bài 1. *Cho văn phạm* $G =$

a) *Cho* G *sinh ra xâu* $u =$

GIÁO TRÌNH MÔ PHỎNG TOÁN

b) G thuộc lớp văn phạm nào?

c) Tìm $L(G)$

Bài 2. Cho văn phạm $G =$

a) Cho G sinh ra xâu $u =$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

b) G thuộc lớp văn phạm nào?

c) Tìm $L(G)$

Bài 3. Cho văn phạm $G =$

a) Cho G sinh ra xâu $u =$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

b) G thuộc lớp văn phạm nào?

c) Tìm $L(G)$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Bài 4. a) Tìm văn phạm G sinh ra xâu $u =$ Chứng minh.

b) G thuộc lớp văn phạm nào?

c) Cho G sinh ra xâu $u =$

TRÌNH MÔ HÌNH TÍNH TOÁN

Bạn học được bằng cách viết ra nhiều tương đương với việc học.

ERIC HOFFER

CHAPTER II. AUTOMATA HỮU HẠN VÀ NGÔN NGỮ CHÍNH QUY

§1. AUTOMATA HỮU HẠN

Trong chương I đã giới thiệu về các khái niệm cơ bản của mô hình tính toán, các mô tả về automata ở mức đơn giản và chưa chính thức. Tại thời điểm này, chúng ta chỉ biết automata là gì và nó có thể được biểu diễn bằng biểu đồ ra sao. Để tiếp tục, chúng ta phải mô tả một cách rõ ràng hơn, đưa ra định nghĩa chính thức và phát triển các kết quả đúng đắn. Chúng ta bắt đầu với một trường hợp đơn giản của biểu đồ tổng quát được giới thiệu ở chương trước, một "***accepter hữu hạn***".

Một automata hữu hạn làm việc theo thời gian rời rạc như tất cả các mô hình tính toán khác. Automata đơn giản này không có bộ nhớ tạm thời nên bị hạn chế về khả năng "ghi nhớ" trong quá trình tính toán. Lượng thông tin hữu hạn được lưu trữ lại trong bộ điều khiển bằng cách đặt bộ điều khiển ở một trạng thái cụ thể. Vì số lượng các trạng thái là hữu hạn, một automata ở mỗi thời điểm, chỉ xử lý được lượng thông tin đầu ra chỉ phụ thuộc vào thông tin vào lúc đó. Các thiết bị như vậy là mô hình của các mạch tổ hợp.

I. Bộ chấp nhận hữu hạn đơn định

Cũng tương tự như tất cả các automata, bộ chấp nhận đơn định cũng có bộ điều khiển trạng thái bên trong, các quy tắc chuyển từ trạng thái này sang trạng thái khác, một số đầu và cách đưa ra quyết định chấp nhận xâu.

1. Định nghĩa

Định nghĩa 1.1.1.

-
-
-
-
-

Cơ chế

- Tại

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Dưới

- Nếu

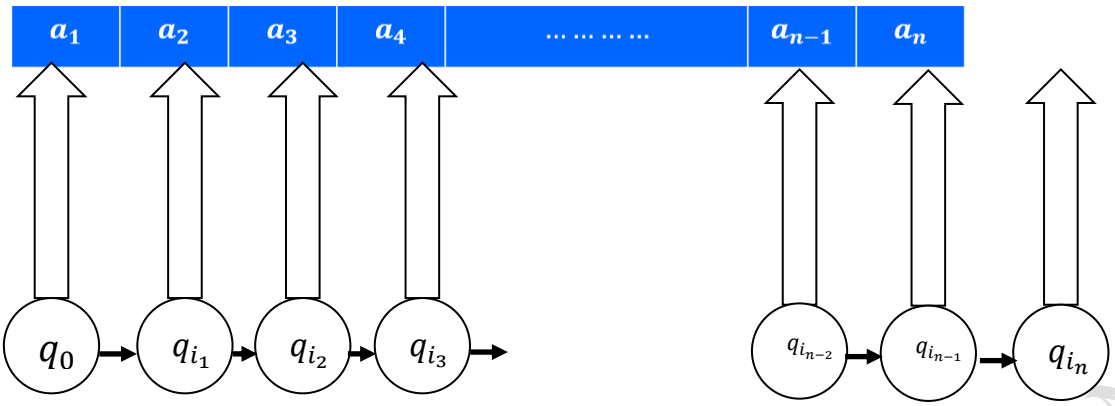
Quá trình đọc xâu u tiếp tục đến khi gặp một trong các trường hợp sau:

- DFA

- DFA

- DFA

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN



Chú ý.

- ✓ ***Tập Q thể hiện các trạng thái ghi nhớ của DFA trong quá trình đoán nhận, vì vậy khả năng ghi nhớ của DFA là hữu hạn. Mặt khác hàm chuyển δ đơn trị nên bước chuyển của automata luôn xác định một cách duy nhất.***
- ✓ ***Nếu $D = Q \times \Sigma$ thì DFA luôn đọc xong xâu u và DFA được gọi là DFA đầy đủ.***

Ví dụ 1.1.2. *Cho DFA $M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_3\})$ trong đó hàm chuyển trạng thái δ xác định như sau: $\delta(q_0, a) = \delta(q_1, a) = \delta(q_2, a) = q_2, \delta(q_0, b) = \delta(q_1, b) = q_1, \delta(q_2, b) = \delta(q_3, a) = \delta(q_3, b) = q_3$.*

a) Vẽ mô hình M đoán nhận các xâu $u_1 = bbaabab, u_2 = bbbbaaa$.

Xâu u_1

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Xâu u_2

b) Viết dãy suy dẫn M đoán nhận các xâu $u_1 = bbaabab, u_2 = bbbbaaa$.

$q_0u_1 =$

nên

$q_0u_2 =$

nên

c) Lập bảng chuyển trạng thái và vẽ đồ thị biểu diễn M.

δ	a	b

d) Tìm $L(M)$ và viết biểu thức chính quy mô tả $L(M)$.

Để

- Đường đi

với

- Đường đi

với

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Biểu thức

e) Xây dựng văn phạm chính quy G tương đương với M. Cho G sinh ra các xâu u_1, u_2.

Do

Do

Do

Do

Do

Do

Do

Vậy

q_0

nên

q_0

Để tiện lợi ta sẽ sử dụng hàm chuyển trạng thái mở rộng từ $\delta^*: Q \times \Sigma^* \longrightarrow Q$. Khi đó đối số thứ hai của hàm δ^* là một xâu thuộc Σ^* chứ không phải là một ký hiệu đơn lẻ thuộc δ và giá trị của hàm δ^* cho chúng ta biết trạng thái của automata sau khi đọc xâu này.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 1.1.3. *Nếu* $\delta(q_0, 1) = q_1, \delta(q_1, 0) = q_2$ *thì ta viết* $\delta^*(q_0, 10) = q_2$.

Về mặt hình thức, ta có thể định nghĩa hàm chuyển trạng thái mở rộng δ^* như sau:

$$\delta^*(q, \lambda) = q$$

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

với mọi $q \in Q, a \in \Sigma$ và $w \in \Sigma^*$.

Ví dụ 1.1.4. *Trong ví dụ 1.1.3 ta có:*

$$\delta^*(q_0, 10) = \delta(\delta^*(q_0, 1), 0) = \delta(\delta(\delta^*(q_0, \lambda), 1), 0) = \delta(\delta(q_0, 1), 0) = \delta(q_1, 0) = q_2$$

2. Ngôn ngữ đoán nhận bởi DFA

Định nghĩa 1.1.5.

$q_0 u =$

trong đó

Định nghĩa 1.1.6.

Định nghĩa 1.1.7.

3. Ý nghĩa của DFA

DFA đóng vai trò là mô hình tiêu chuẩn để máy tính thực thi các tác vụ nhận diện mẫu vì những lý do sau:

- *Tính dự đoán tuyệt đối: Với một chuỗi đầu vào w, DFA luôn đi qua một lộ trình trạng thái duy nhất. Điều này loại bỏ hoàn toàn sự mơ hồ và việc phải "thử sai" (backtracking).*
- *Hiệu năng tối ưu $(O(n))$: DFA chỉ cần đọc chuỗi đầu vào đúng một lần từ trái sang phải. Thời gian xử lý tỉ lệ thuận trực tiếp với độ dài chuỗi, không phụ thuộc vào độ phức tạp của mẫu (pattern).*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- *Tiết kiệm tài nguyên: Vì tại mỗi thời điểm máy chỉ ở đúng một trạng thái, bộ nhớ cần thiết để chạy một DFA là cực kỳ thấp (chỉ cần một biến để lưu ID trạng thái hiện tại).*

4. Ứng dụng của DFA

DFA xuất hiện ở mọi nơi mà tốc độ và sự chính xác là ưu tiên hàng đầu:

a) Phân tích từ vựng trong Trình biên dịch (Lexical Analysis)

Đây là ứng dụng kinh điển nhất. Khi bạn viết code (C++, Python...), bước đầu tiên của trình biên dịch là biến các chuỗi ký tự thành các "Token" (từ khóa, tên biến, toán tử).

Ví dụ 1.1.8. *Nhận diện từ khóa int, if, while. Các bộ quét từ vựng (như Lex hoặc Flex) sẽ chuyển các quy tắc này thành một DFA khổng lồ để quét mã nguồn với tốc độ chóng mặt.*

b) Thiết kế phần cứng và Mạch số

Các kỹ sư điện tử sử dụng DFA dưới tên gọi Máy trạng thái hữu hạn (FSM) để điều khiển các thiết bị:

- *Bộ điều khiển đèn giao thông*: Trạng thái chuyển từ $Xanh \longrightarrow Vàng \longrightarrow Đỏ$ dựa trên bộ đếm thời gian.
- *Máy bán hàng tự động*: Trạng thái thay đổi dựa trên số tiền bạn nạp vào cho đến khi đủ để xuất hàng.
- *CPU*: Các đơn vị điều khiển bên trong chip xử lý thực chất là các DFA điều phối việc lấy lệnh và thực thi lệnh.

c) Giao thức truyền tin (Network Protocols)

Các giao thức mạng như TCP hoạt động dựa trên mô hình DFA để đảm bảo sự ổn định của kết nối:

- Khi bạn gửi yêu cầu kết nối, máy chuyển từ trạng thái CLOSED sang SYN_SENT.
- Nếu nhận được phản hồi, nó chuyển sang ESTABLISHED.
- Mọi sự kiện (nhận gói tin, timeout) đều được quản lý chặt chẽ bởi các bước chuyển trạng thái của DFA.

d) Hệ thống lọc và tìm kiếm (String Matching)

- *Công cụ grep trong Linux*: Khi bạn tìm kiếm một từ trong file văn bản lớn, grep thường chuyển biểu thức tìm kiếm của bạn thành DFA để đảm bảo tốc độ quét nhanh nhất có thể.
- *Kiểm duyệt nội dung*: Các hệ thống lọc từ cấm trên mạng xã hội hoặc lọc thư rác thường dùng DFA để đối soát hàng triệu tin nhắn mỗi giây.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

II. Bộ chấp nhận hữu hạn không đơn định

1. Định nghĩa và ngôn ngữ được đoán nhận

Định nghĩa 1.2.1.

-
-
-
-
-

Định nghĩa NFA khác với DFA ở ba điểm sau:

- Giá trị của hàm chuyển trạng thái không đơn thuần là một phần tử của Q mà là một tập con của Q. Tập hợp con này xác định tập hợp tất cả các trạng thái có thể đến được bởi hàm chuyển.
- Cho phép λ là đối số thứ hai của hàm chuyển trạng thái δ. Điều này có nghĩa là NFA có thể thực hiện bước chuyển mà không cần sử dụng đầu vào. Mặc dù chúng ta vẫn giả định rằng đầu đọc chỉ có thể di chuyển sang bên phải, nhưng có thể nó đứng yên trên một số bước di chuyển.
- Cuối cùng, trong một NFA, tập $\delta(q,a)$ có thể trống, nghĩa là không có sự chuyển đổi nào được xác định cho tình huống cụ thể này.

Nhận xét. DFA là một trường hợp đặc biệt của NFA vì $Q \in 2^Q$.

Định nghĩa 1.2.2.

Ví dụ 1.2.3. *Cho $FA\ M = (\{q_0, q_1, q_2, q_3\}, \{0,1\}, \delta, q_0, \{q_3\})$ trong đó $\delta(q_0,0) = \{q_0, q_1\}$, $\delta(q_0,1) = \{q_0\}, \delta(q_1,1) = \{q_1, q_2\}, \delta(q_2,0) = \delta(q_2,1) = \{q_3\}$.*

a) Chứng minh M là NFA.

M là

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

-

-

b) Cho M đoán nhận xâu $u_1 = 1010110010$ và $u_2 = 100101101$.

$q_0u_1 =$

$q_0u_1 =$

$q_0u_1 =$

Vậy

$q_0u_2 =$

Vậy

c) Lập bảng chuyển trạng thái và vẽ đồ thị biểu diễn M.

δ	0	1

d) Lập bảng chuyển trạng thái và vẽ đồ thị biểu diễn M.

Để

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

tức là

Biểu thức

e) Tìm DFA M_1 tương đương với M. Vẽ đồ thị chuyển của M_1.

δ_1	0	1

Do

Vậy

Đồ thị

f) Cho M_1 đoán nhận xâu u_1, u_2.

$r_0 u_1 =$

nên

$r_0 u_2 =$

nên

g) Xây dựng văn phạm chính quy G tương đương với M và cho G sinh ra các xâu u_1, u_2.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Do

Do

Do

Vậy

q_0

Vậy

q_0

Vậy

2. Ý nghĩa của NFA

Trong khi DFA (Đơn định) dễ cài đặt cho máy tính, thì NFA lại có ý nghĩa to lớn về mặt tư duy và thiết kế:

- *Sự đơn giản trong thiết kế: NFA cho phép máy tính ở nhiều trạng thái cùng một lúc hoặc đi theo nhiều nhánh "dự đoán". Điều này giúp sơ đồ NFA thường nhỏ gọn và dễ hiểu hơn DFA rất nhiều lần đối với cùng một ngôn ngữ.*
- *Mô hình hóa sự "Dự đoán" (Guessing): NFA đại diện cho tư duy: "Nếu có một đường đi dẫn đến đích, tôi sẽ chấp nhận". Nó giúp giải quyết các bài toán mà chúng ta không biết chắc chắn lựa chọn nào là đúng ngay từ đầu.*
- *Tính tương đương toán học: Một định lý quan trọng là Mọi NFA đều có thể chuyển đổi thành DFA. Điều này có nghĩa là con người cứ việc thiết kế bằng NFA cho dễ, rồi để thuật toán tự động chuyển nó sang DFA cho máy chạy.*

3. Ứng dụng của NFA

NFA không chỉ là lý thuyết suông, nó nằm dưới "nắp ca-pô" của nhiều công nghệ bạn dùng hàng ngày:

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

a) Xây dựng bộ máy tìm kiếm Biểu thức chính quy (Regex Engine)

Đây là ứng dụng phổ biến nhất. Khi bạn viết một Regex phức tạp (như kiểm tra email hay số điện thoại), hầu hết các thư viện lập trình sẽ:

- Chuyển Regex đó thành một NFA (thường dùng thuật toán Thompson).
- Sau đó, máy tính sẽ mô phỏng NFA này bằng cách theo dõi tất cả các trạng thái có thể có cùng lúc khi đọc văn bản.

b) Kiểm chứng phần mềm và hệ thống (Model Checking)

Trong các hệ thống quan trọng (như phần mềm điều khiển tên lửa, xe tự lái), người ta dùng NFA để mô hình hóa mọi khả năng có thể xảy ra của hệ thống. NFA giúp kiểm tra xem có bất kỳ "nhánh" nào dẫn đến lỗi nghiêm trọng hay không.

c) Phân tích giao thức mạng (Network Protocols)

Các giao thức truyền tin như TCP hoặc các hệ thống phát hiện xâm nhập (IDS) sử dụng NFA để theo dõi các gói tin. Vì các gói tin có thể đến sai thứ tự hoặc bị mất, NFA cho phép hệ thống duy trì nhiều kịch bản trạng thái để xử lý sự không chắc chắn này.

d) Thiết kế bộ phân tích từ vựng (Lexical Analyzers)

Trong các trình biên dịch (như GCC cho C++, hay trình thông dịch Python), NFA được dùng để gom nhóm các ký tự thành các từ khóa (keywords), tên biến (identifiers). Người ta viết quy tắc bằng NFA rồi mới dùng công cụ (như Lex/Flex) để nén nó thành DFA tối ưu cho tốc độ.

III. Biểu diễn bộ chấp nhận hữu hạn

Hàm chuyển trạng thái là bộ phận quan trọng của FA. Cho một FA cũng là cho một hàm chuyển trạng thái. Hàm chuyển trạng thái có thể biểu diễn bởi bảng chuyển hoặc đồ thị chuyển.

1. Biểu diễn FA bằng bảng chuyển trạng thái

Cho bộ chấp nhận hữu hạn $M = (\{q_0, q_1, \ldots, q_m\}, \{a_1, a_2, \ldots, a_n\}, \delta, q_0, F)$. Khi đó hàm chuyển trạng thái δ có thể cho bởi bảng trong đó hàng i cột j là ô trống nếu $\delta(q_i, a_j)$ không xác định.

Trạng thái	Ký hiệu vào			
	a_1	a_2	.....	a_n
q_1	$\delta(q_1, a_1)$	$\delta(q_1, a_2)$	.....	$\delta(q_1, a_n)$
q_2	$\delta(q_2, a_1)$	$\delta(q_2, a_2)$	.....	$\delta(q_2, a_n)$
.....	.....	.....	.....	.....
q_m	$\delta(q_m, a_1)$	$\delta(q_m, a_2)$	.....	$\delta(q_m, a_n)$

Chú ý.

GIÁO TRÌNH MÔ HÌNH TOÁN

- ✓ ***Nếu FAM cho bảng chuyển trạng thái thì nếu chỉ rõ tập trạng thái kết thúc F thì sẽ xác định được FAM.***
- ✓ ***Bảng chuyển trạng thái cho một gợi ý về cấu trúc dữ liệu để mô tả FA.***
- ✓ ***Một FA có thể dễ dàng hiện thực thành một chương trình máy tính nhờ một dãy lệnh if.***

Ý nghĩa của biểu diễn FA bằng bảng chuyển

- *Tính đầy đủ và chính xác: Bảng bắt buộc bạn phải liệt kê mọi tổ hợp có thể giữa trạng thái và ký tự nhập vào. Điều này đảm bảo không có bước chuyển nào bị bỏ sót (đặc biệt quan trọng đối với DFA).*
- *Cấu trúc dữ liệu cho máy tính: Máy tính không "nhìn" được hình vẽ, nhưng nó cực kỳ giỏi xử lý ma trận và mảng. Bảng chuyển chính là một mảng hai chiều mà máy tính có thể tra cứu trong thời gian $O(1)$.*

Ứng dụng của biểu diễn FA bằng bảng chuyển

- × Lập trình và Thực thi: Khi bạn viết mã nguồn cho một bộ quét từ vựng (Lexer), bạn sẽ lưu bảng chuyển này vào bộ nhớ. Mỗi lần nhận một ký tự, máy chỉ việc tra bảng:

```
Trạng thái_mới = Bang[Trạng thái_hien_tai][Ky_tu_nhap]
```

- × Chuyển đổi NFA sang DFA: Các thuật toán như Subset Construction (Xây dựng tập con) thực hiện hoàn toàn trên bảng vì nó giúp quản lý các tập hợp trạng thái một cách hệ thống, tránh nhầm lẫn so với việc vẽ tay.

2. Biểu diễn FA bằng đồ thị chuyển trạng thái

Cho

.....

.....

.....

.....

.....

.....

.....

.....

Nhận xét.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- ***Quá trình đoán nhận xâu $u = a_1a_2\dots a_n$ chính là một đường đi từ đỉnh đầu q_0 đến đỉnh q_{i_n} qua các cạnh có nhãn lần lượt là $a_1, a_2, \dots, a_n$.***
- ***$L(M)$ chính là tập hợp các xâu trên đường đi từ đỉnh đầu đến đỉnh kết thúc.***

Hàm chuyển trạng thái mở rộng của NFA có thể xác định đệ quy tương tự như đối với DFA nhưng thường sẽ không rõ ràng. Một định nghĩa được đánh giá cao hơn là dựa vào đồ thị chuyển.

Định nghĩa 1.3.1. Cho NFA $M = (Q, \Sigma, \delta, q_0, F)$. Hàm chuyển trạng thái mở rộng δ^* xác định như sau: Với mỗi $q_i, q_j \in Q, u \in \Sigma^*$ thì $\delta(q_i, u)$ chứa q_j nếu có một đường đi từ đỉnh nhãn q_i sang đỉnh có nhãn q_j với nhãn đường đi là u.

- ***Khi đó ngôn ngữ được đoán nhận bởi M là***

$$L(M) = \{u \in \Sigma^* | \delta^*(q_0, u) \cap F \neq \emptyset\}$$

Ý nghĩa của biểu diễn FA bằng đồ thị :

- *Trực quan hóa dòng chảy: Giúp con người dễ dàng hình dung lộ trình của một chuỗi ký tự từ trạng thái bắt đầu đến đích. Bạn có thể thấy ngay các vòng lặp và các nhánh rẽ.*
- *Phát hiện lỗi logic nhanh: Bằng mắt thường, ta có thể nhận ra các "trạng thái chết" (trạng thái không bao giờ đến được đích) hoặc các thiếu sót trong luồng xử lý.*
- *Tính trừu tượng cao: Thích hợp để thảo luận ý tưởng, thiết kế hệ thống ở mức khái niệm.*

Ứng dụng của biểu diễn FA bằng đồ thị chuyển

- Thiết kế giao diện (UI/UX): Mô tả các trạng thái của màn hình (ví dụ: $\text{Đang tải} \longrightarrow \text{Thành công} \longrightarrow \text{Hiển thị dữ liệu}$).
- Đào tạo và giáo dục: Là công cụ tốt nhất để dạy về logic và lý thuyết tính toán.

3. Sự kết hợp giữa đồ thị chuyển và bảng chuyển

Trong quy trình làm việc của một kỹ sư Khoa học máy tính, hai công cụ này bổ trợ cho nhau theo chu trình:

- *Đồ thị*: Dùng để phác thảo ý tưởng ban đầu (Human-friendly).
- *Bảng*: Dùng để chuẩn hóa logic và kiểm tra tính đầy đủ (Verification).
- *Mã nguồn*: Chuyển bảng vào mảng/hàm trong code để chạy (Machine-friendly).

IV. Sự tương đương giữa DFA và NFA

Việc chuyển đổi từ Máy tự động hữu hạn không đơn định (NFA) sang Máy tự động hữu hạn đơn định (DFA) là một quy trình kỹ thuật cốt lõi trong lý thuyết tính toán. Về mặt bản chất, đây là quá trình biến một mô hình có tính "dự đoán/lựa chọn" thành một mô hình "quyết định/thực thi".

1. Phương pháp chuyển từ NFA thành DFA

Định nghĩa 1.4.1.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Do mỗi DFA cũng là NFA nên lớp ngôn ngữ được đoán nhận bởi DFA sẽ được đoán nhận bởi NFA. Ta xét trường hợp ngược lại.

Định lý 1.4.2.

Giả sử $M = (Q, \Sigma, \delta, q_0, F)$ đoán nhận L. Xây dựng DFA $M' = (Q', \Sigma, \delta', q_0', F')$ như sau:

- $Q' = 2^Q$.
- F' là tập mọi trạng thái trong Q' có chứa một trạng thái cuối nào đó của M.
- Một phần tử trong Q' là tập hợp $\{q_1, q_2, \dots, q_k\}$ được ký hiệu là $[q_1, q_2, \dots, q_k]$ trong đó $q_i \in Q$.
- Trạng thái đầu $q_0' = [q_0]$.
- Hàm chuyển trạng thái δ' được thành lập theo công thức:

$$\delta'([q_1, q_2, \dots q_k], a) = \bigcup_{i=1}^{k} \delta(q_i, a)$$

Hệ quả 1.4.3.

Chú ý. Trong 2^Q thường có nhiều phần tử không thể có đường đi từ $[q_0]$ nên không cần đưa chúng vào Q'. Do đó nên xuất phát từ $[q_0]$ rồi từng bước thêm các trạng thái mới, nếu các trạng thái này là kết quả của hàm chuyển áp dụng lên các trạng thái trước đó.

Ví dụ 1.4.4.

a) *Xây dựng FA M đoán nhận ngôn ngữ trên bảng chữ $\{a, b\}$ có hậu tố là xâu abb.*

Vậy

δ_1	a	b

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

b) Xây dựng $DFA\ M_1$ tương đương với M và vẽ đồ thị chuyển biểu diễn M_1. Cho M_1 đoán nhận xâu $u = abbababb$.

Do

Vậy

Đồ thị

$r_0u =$

Vậy

c) Xây dựng văn phạm chính quy G tương đương với M. Cho G sinh ra xâu u.

Do

Do

Do

Do

Vậy

$q_0 =$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Vậy

Ví dụ 1.4.5.

a) Xây dựng FA M đoán nhận ngôn ngữ trên bảng chữ nhị phân có chứa xâu con 000.

Vậy

b) Xây dựng DFA M_1 tương đương với M và vẽ đồ thị chuyển biểu diễn M_1. Cho M_1 đoán nhận xâu $u =$ 011000010.

δ_1	0	1

Do

Vậy

Đồ thị

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

$r_0 u =$

Vậy

c) *Xây dựng văn phạm chính quy G tương đương với M. Cho G sinh ra xâu u.*

Do

Do

Do

Do

Do

Vậy

$q_0 \Longrightarrow$

Vậy

Ví dụ 1.4.6.

a) *Xây dựng FAM đoán nhận ngôn ngữ trên bảng chữ nhị phân luôn chứa xâu con 000 và nếu có xuất hiện ký hiệu 0 thì ký hiệu 0 này phải thuộc xâu con có đúng ba chữ số 0 và không chứa xâu con 0000.*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Vậy

b) Xây dựng DFA M_1 tương đương với M và vẽ đồ thị chuyển biểu diễn M_1. Cho M_1 đoán nhận xâu $u =$ 11000010 và $v =$ 000110001.

δ_1	0	1

Do

Vậy

$r_0u =$

Ở

$r_0v =$

Vậy

Đồ thị

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

c) *Xây dựng văn phạm chính quy G tương đương với M. Cho G sinh ra xâu u.*

Do

Do

Do

Do

Do

Do

Vậy

$q_0 \Rightarrow$

Vậy

2. Ý nghĩa của việc chuyển từ NFA sang DFA

Việc chuyển đổi từ NFA sang DFA mang lại ba ý nghĩa lớn về mặt toán học và kỹ thuật:

- *Chứng minh tính tương đương: Quy trình này chứng minh rằng tính "không đơn định" không làm tăng sức mạnh tính toán của máy tự động hữu hạn. Bất kỳ ngôn ngữ nào được chấp nhận bởi một NFA thì cũng luôn có một DFA tương ứng chấp nhận nó.*
- *Loại bỏ sự mơ hồ: NFA cho phép tồn tại nhiều lựa chọn cho cùng một đầu vào, điều này khiến máy tính khó xử lý trực tiếp. Chuyển sang DFA giúp xác định duy nhất một lộ trình trạng thái, giúp quá trình kiểm tra chuỗi trở nên tường minh.*
- *Tối ưu hóa tốc độ thực thi: Trong khi NFA cần theo dõi nhiều nhánh trạng thái cùng lúc (gây tốn bộ nhớ và thời gian xử lý), DFA chỉ cần một lần quét duy nhất từ trái sang phải với độ phức tạp thời gian là $O(n)$.*

3. Ứng dụng của việc chuyển từ NFA sang DFA trong khoa học máy tính

Quy trình chuyển đổi này (thường thông qua thuật toán Subset Construction) là "trái tim" của nhiều công cụ phần mềm hiện đại:

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

a) Xây dựng bộ máy xử lý Biểu thức chính quy (Regex Engine)

Đây là ứng dụng phổ biến nhất. Hầu hết các thư viện Regex (như trong Python, Java, hay công cụ grep) hoạt động theo quy trình:

- Người dùng viết Regex.
- Hệ thống chuyển Regex thành NFA bằng thuật toán Thompson (Dễ thiết kế máy).
- Hệ thống tiếp tục chuyển NFA thành DFA để tìm kiếm trên văn bản với tốc độ nhanh nhất.

b) Phân tích từ vựng trong Trình biên dịch (Lexical Analysis)

Các trình biên dịch ngôn ngữ lập trình sử dụng ứng dụng này để nhận diện các từ khóa (tokens).

- Các quy tắc nhận diện từ khóa (như if, while, tên biến) được định nghĩa bằng các NFA riêng lẻ.
- Sau đó, tất cả các NFA này được gộp lại và chuyển thành một DFA duy nhất để quét toàn bộ file mã nguồn chỉ trong một lượt duy nhất.

c) Kiểm chứng hệ thống (Formal Verification)

Trong việc kiểm tra logic của các hệ thống an toàn (như phần mềm điều khiển máy bay hay thiết bị y tế):

- Các kịch bản lỗi thường được mô tả dưới dạng NFA để bao hàm mọi khả năng xảy ra.
- Việc chuyển sang DFA giúp các công cụ kiểm chứng tự động duyệt qua toàn bộ không gian trạng thái một cách hệ thống để đảm bảo không có nhánh nào dẫn đến thảm họa.

Quy trình ứng dụng thực tế:

$\acute{Y}\ tưởng\ (Regex) \longrightarrow Mô\ hình\ hóa\ (NFA) \longrightarrow Chuyển\ đổi\ (Subset\ Construction)$

$\longrightarrow Thực\ thi\ hiệu\ năng\ cao\ (DFA).$

BÀI TẬP BUỔI 3

Bài 1. *Cho FA* $M =$

a) Cho M *đoán nhận xâu*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

nên

b) Cho M đoán nhận xâu

nên

c) Cho M đoán nhận xâu

nên

d) Tìm ngôn ngữ do M đoán nhận

Bài 2. Đơn định FA M =

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

§2. AUTOMATA HỮU HẠN VÀ NGÔN NGỮ CHÍNH QUY

I. Biểu thức chính quy

1. Định nghĩa và ví dụ

Biểu thức chính quy (Regular Expression - Regex) là một chuỗi các ký tự đặc biệt dùng để mô tả một bộ quy tắc tìm kiếm. Trong lý thuyết tính toán, Regex là cách biểu diễn đại số cho ngôn ngữ chính quy, có sức mạnh tương đương với máy tự động hữu hạn (FA).

Định nghĩa 2.1.1.

-

-

-

Ví dụ 2.1.2. *Với bảng chữ* $\Sigma = \{a, b\}$ *thì* $(ab + a)^*(a + \lambda)(aba + \emptyset)^*$ *là biểu thức chính quy.*

Biểu thức chính quy có thể được sử dụng để mô tả một số ngôn ngữ đơn giản. Nếu r là một biểu thức chính quy, chúng ta sẽ ký hiệu $L(r)$ là ngôn ngữ liên mô tả bởi r.

Định nghĩa 2.1.3.

-

-

-

-

Bốn quy tắc cuối cùng của định nghĩa này được sử dụng một cách đệ quy để đưa $L(r)$ về các thành phần đơn giản hơn. Quy tắc đầu tiên là điều kiện để kết thúc đệ quy này. Để biết một ngôn ngữ mà một biểu thức biểu thị, ta áp dụng các quy tắc này lặp đi lặp lại.

Ví dụ 2.1.4. *Hãy biểu diễn các ngôn ngữ có biểu thức chính quy sau và mô tả ngôn ngữ này:*

a) $(0 + 1)^*$

Vậy

b) $a^*(a + b)$

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

c) $(a+b)^*aba$

Vậy

d) $(a+b)^*(a+ab)$

Vậy

e) $(0+01)^*$

Vậy

f) $(00)^*(111)^*$

Vậy

Ví dụ 2.1.5*. Tìm các biểu thức chính quy mô tả các ngôn ngữ sau:*

a) L_1 = *tập các xâu nhị phân có chứa xâu con* 01.

$L_1 =$

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

b) L_2 là tập các xâu có dạng $a^n b^m$ với $n \geq 3$, m là số lẻ.

$L_2 =$ .....

.....

nên .....

2. Ý nghĩa của Biểu thức chính quy

- *Sự ngắn gọn và súc tích: Một dòng Regex có thể thay thế cho hàng chục dòng lệnh if-else phức tạp. Nó cho phép bạn mô tả các cấu trúc văn bản lặp lại một cách cực kỳ gọn nhẹ.*
- *Tính trừu tượng hóa cao: Regex không quan tâm đến nội dung cụ thể mà quan tâm đến cấu trúc của dữ liệu. Ví dụ: Bạn không tìm số "090...", bạn tìm "một chuỗi bắt đầu bằng số 0 và có 10 chữ số".*
- *Cầu nối giữa con người và máy tính: Regex là ngôn ngữ "khai báo". Bạn chỉ cần nói cho máy biết bạn muốn tìm cái gì, còn việc tìm như thế nào (chạy qua các trạng thái DFA/NFA nào) là do bộ máy Regex tự xử lý.*
- *Tính chuẩn hóa: Hầu hết các ngôn ngữ lập trình (Python, Java, JavaScript, C#...) và các công cụ hệ thống (Linux, SQL) đều hỗ trợ Regex với cú pháp gần như tương đồng.*

3. Ứng dụng thực tế của Regex

a) Kiểm tra tính hợp lệ của dữ liệu (Data Validation)

Đây là ứng dụng phổ biến nhất trong lập trình web và ứng dụng.

Ví dụ 2.1.6. *Kiểm tra xem người dùng có nhập đúng định dạng Email, số điện thoại, hoặc mật khẩu có đủ độ mạnh (bao gồm chữ hoa, chữ thường và ký số) hay không.*

b) Tìm kiếm và thay thế nâng cao (Find & Replace)

Trong các trình soạn thảo văn bản (Notepad++, VS Code) hoặc các công cụ xử lý văn bản (Word): Bạn có thể tìm tất cả các ngày tháng có định dạng dd/mm/yyyy và chuyển chúng thành yyyy-mm-dd chỉ bằng một lệnh Replace duy nhất nhờ tính năng Capture Groups của Regex.

c) Phân tích từ vựng trong Trình biên dịch (Lexical Analysis)

Như đã thảo luận ở các phần trước, các trình biên dịch sử dụng Regex để nhận diện các "Token": Xác định đâu là từ khóa (if, else), đâu là hằng số số thực (3.14), đâu là tên biến. Regex là công cụ đầu tiên "đọc" mã nguồn của bạn.

GIÁO TRÌNH MÔ HÌNH TOÁN

d) Trích xuất dữ liệu (Data Scraping)

Khi cần lấy thông tin từ một đống dữ liệu hỗn độn hoặc từ mã nguồn HTML của một trang web: Regex giúp lọc ra danh sách các liên kết (links), giá sản phẩm, hoặc mã chứng khoán từ các trang tin tức một cách tự động.

e) Quản trị hệ thống và An ninh mạng

- Phân tích Log: Các kỹ sư hệ thống dùng Regex để quét qua hàng triệu dòng log máy chủ nhằm tìm ra các dấu hiệu bị tấn công (như SQL Injection hoặc Brute Force).
- Cấu hình Server: Các máy chủ web như Nginx hoặc Apache dùng Regex để điều hướng (rewrite) URL, giúp tạo ra các đường dẫn thân thiện với người dùng và SEO.

4. Mối quan hệ giữa Regex và Automata

Để thực thi một biểu thức Regex, máy tính thường trải qua quy trình sau:

- Chuyển Regex sang NFA: Sử dụng thuật toán Thompson.
- Chuyển NFA sang DFA: Để tối ưu tốc độ tìm kiếm.
- Thực thi: Máy tính chạy chuỗi văn bản qua DFA đó để trả về kết quả Đúng/Sai.

II. Ngôn ngữ chính quy

1. Định nghĩa và ví dụ

Định nghĩa 2.2.1.

Ví dụ 2.2.2. *Chứng minh rằng ngôn ngữ* $L = \{a^n b^m | n, m \geq 1\}$ *là ngôn ngữ chính quy.*

Do

nên

Định lý 2.2.3.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 2.2.4. *Chỉ ra các NFA đoán nhận các ngôn ngữ có biểu thức chính quy mô tả là* $\emptyset, \lambda, a, a(a+b)^*ab$ *trên bảng chữ* $\{a, b\}$.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định lý 2.2.5.

Ví dụ 2.2.6. *Cho L tập các xâu nhị phân luôn chứa cặp 00 và có tiền tố là 01.*

a) *Chứng minh rằng L là ngôn ngữ chính quy.*

L

L

b) *Chỉ ra biểu thức chính quy r có biểu thức mô tả là L.*

L

2. Ý nghĩa của việc định nghĩa ngôn ngữ chính quy thông qua FA

- *Tính khả thi về mặt tính toán (Computational Feasibility): Một ngôn ngữ được gọi là "chính quy" nếu và chỉ nếu có một FA chấp nhận nó. Điều này có nghĩa là ngôn ngữ đó có thể được xử lý bởi một cỗ máy có bộ nhớ hữu hạn và cố định, không phụ thuộc vào độ dài chuỗi đầu vào.*
- *Tính đơn định và hiệu quả: Khi định nghĩa bằng DFA, ta có một quy trình xử lý cực kỳ nhanh với độ phức tạp thời gian là $O(n)$. Máy tính không bao giờ phải "đoán" hay quay lui, giúp việc kiểm tra một chuỗi có thuộc ngôn ngữ hay không trở nên tối ưu tuyệt đối.*
- *Cơ sở để chứng minh các tính chất ngôn ngữ: Nhờ FA, các nhà khoa học có thể sử dụng các công cụ toán học (như Bổ đề bơm - Pumping Lemma) để chứng minh một ngôn ngữ có chính quy hay không, hoặc chứng minh các tính chất đóng (hợp, giao, bù) của ngôn ngữ chính quy một cách dễ dàng.*
- *Sự thống nhất giữa các mô hình: FA cung cấp một "máy thực thi" cho các khái niệm khác. Biểu thức chính quy (Regex) là cách con người viết, còn FA là cách máy tính hiểu.*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Việc định nghĩa qua FA giúp kết nối đại số (Regex), văn phạm (Regular Grammar) và cơ học (Automata).

3. Ứng dụng thực tế

Khi một ngôn ngữ được định nghĩa và triển khai dưới dạng một FA, nó mở ra các ứng dụng thực tiễn sau:

a) Phân tích từ vựng trong trình biên dịch (Lexical Analysis)

Đây là ứng dụng quan trọng nhất. Các ngôn ngữ lập trình định nghĩa các "từ" của mình (như số thực, tên biến, từ khóa) là các ngôn ngữ chính quy.

Ví dụ. Một FA sẽ được thiết kế để nhận diện đâu là số thực hợp lệ (ví dụ: 3.14, −0.5). Nếu chuỗi đầu vào đi đến trạng thái kết thúc của FA, trình biên dịch sẽ xác nhận đó là một Token hợp lệ.

b) Thiết kế giao thức mạng và truyền thông

Các ngôn ngữ chính quy qua FA dùng để mô tả các quy tắc giao tiếp.

Ứng dụng: Trong giao thức TCP, các trạng thái như LISTEN, SYN_SENT, ESTABLISHED tạo thành một FA. Ngôn ngữ của FA này chính là tập hợp các trình tự gói tin hợp lệ. Nếu một chuỗi gói tin không tuân theo lộ trình của FA, nó sẽ bị coi là lỗi bảo mật hoặc lỗi kết nối.

c) Kiểm tra và xác thực dữ liệu (Input Validation)

Mọi hệ thống đăng ký web đều sử dụng FA (ẩn dưới dạng Regex) để định nghĩa "ngôn ngữ" của các trường nhập liệu.

Ứng dụng: Kiểm tra định dạng Email, mật khẩu, số thẻ tín dụng. FA giúp máy tính phản hồi ngay lập tức xem dữ liệu có khớp với mẫu hay không.

d) Thiết kế Hệ thống nhúng và Phần cứng

Trong các chip vi xử lý, các đơn vị điều khiển thực chất là các FA phần cứng.

Ứng dụng: Điều khiển thang máy, máy giặt, hay các mạch logic trong CPU. "Ngôn ngữ" ở đây là các chuỗi tín hiệu điện đầu vào, và FA đảm bảo hệ thống phản ứng chính xác với từng chuỗi tín hiệu đó.

e) Xử lý văn bản và Tìm kiếm (Pattern Matching)

Các bộ lọc thư rác (Spam filters) hoặc công cụ tìm kiếm sử dụng FA để quét nhanh qua hàng tỷ ký tự.

Ứng dụng: Tìm kiếm các mẫu virus trong file (Antivirus), lọc các từ nhạy cảm trên mạng xã hội. Vì FA xử lý theo thời gian thực (linear time), nó cực kỳ hiệu quả cho các hệ thống dữ liệu lớn.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

III. Văn phạm chính quy phải và ngôn ngữ chính quy

Mối liên hệ giữa văn phạm chính quy và Automata hữu hạn là một trong những cột trụ quan trọng nhất của lý thuyết tính toán. Về cơ bản, chúng là hai mặt của một đồng xu: một bên là cách tạo ra ngôn ngữ và một bên là cách nhận diện ngôn ngữ đó.

1. Văn phạm chính quy và automata hữu hạn

Định lý 2.3.1.

Xây dựng NFA $M = (Q, T, \delta, q_0, F)$ đoán nhận ngôn ngữ $L(G)$ như sau:

- Thêm vào ký hiệu mới E không thuộc V ∪ T.
- $q_0 = S$.
- $F = \{E\}$ nếu không có sản xuất $S \longrightarrow \lambda$ và $F = \{S, E\}$ nếu có sản xuất $S \rightarrow \lambda$.
- Nếu $A \longrightarrow uB$ là một sản xuất trong P với $u = a_1 a_2 \dots a_k$ với $A, B \in V, u \in T^*$ thì thêm vào $k-1$ trạng thái mới $\overline{a_1}, \overline{a_2}, \dots, \overline{a_{k-1}}$ và $\delta(A, a_1) = \overline{a_1}, \delta(\overline{a_1}, a_2) = \overline{a_2}, \dots, \delta(\overline{a_{k-2}}, a_{k-1}) = \overline{a_{k-1}}, \delta(\overline{a_{k-1}}, a_k) = B$.
- Nếu $A \longrightarrow u$ là một sản xuất trong P với $u = a_1 a_2 \dots a_k$ với $A, B \in V, u \in T^*$ thì thêm vào $k-1$ trạng thái mới $\overline{a_1}, \overline{a_2}, \dots, \overline{a_{k-1}}$ và $\delta(A, a_1) = \overline{a_1}, \delta(\overline{a_1}, a_2) = \overline{a_2}, \dots, \delta(\overline{a_{k-2}}, a_{k-1}) = \overline{a_{k-1}}, \delta(\overline{a_{k-1}}, a_k) = E$.
- $Q = V \cup \{E\} \cup \{\text{các trạng thái mới thêm vào}\}$

Khi đó ta có thể chứng minh được M đoán nhận ngôn ngữ $L(G)$.

Ví dụ 2.3.2. *Cho văn phạm chính quy* $G = (\{S, A\}, \{0,1\}, S, \{S \longrightarrow 0A, A \rightarrow 01S|1\})$

a) *Tìm* $L(G)$ *và biểu thức chính quy* r *mô tả ngôn ngữ* $L(G)$.

S

Vậy

b) *Xây dựng NFA* M *tương đương với* G *và vẽ đồ thị chuyển của* M.

S

A

A

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Vậy

Đồ thị

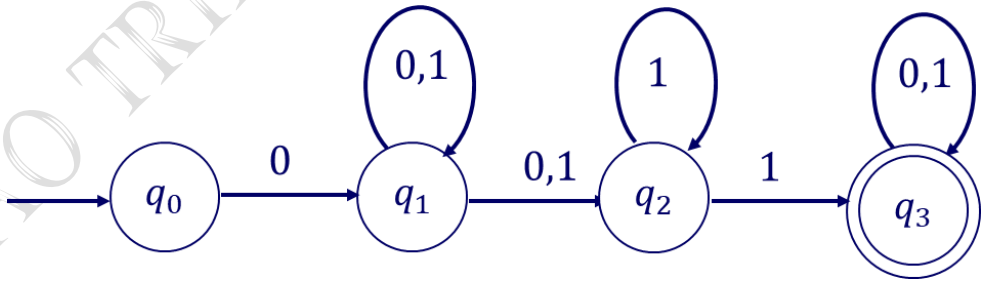
Định lý 2.3.3.

Giả sử $M = (\{q_0, q_1, \dots, q_n\}, \{a_1, a_2, \dots, a_m\}, \delta, q_0, F)$ là một DFA đoán nhận L. Xây dựng văn phạm chính quy phải $G = (\{q_0, q_1, \dots, q_n\}, \{a_1, a_2, \dots, a_m\}, q_0, P)$ sinh ra ngôn ngữ L với tập sản xuất P xây dựng như sau:

- Với mỗi $\delta(q_i, a_j) = q_k$, thêm vào P sản xuất $q_i \longrightarrow a_j q_k$.
- Nếu $q_k \in F$ thì thêm vào P một sản xuất nữa $q_i \longrightarrow \lambda$.

Khi đó có thể chứng minh được $L(G) = L(M)$ hay G sinh ra ngôn ngữ L.

Ví dụ 2.3.4*. Cho FA M có đồ thị chuyển như sau:*



a) Mô tả $L(M)$ và viết biểu thức chính quy r mô tả ngôn ngữ $L(M)$.

Biểu thức

b) Xây dựng văn phạm chính quy G sinh ra ngôn ngữ $L(M)$.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Vậy

Định lý 2.3.5.

Ví dụ 2.3.6. *Tìm văn phạm chính quy G sinh ra ngôn ngữ $L = \{0^n 1^m | n + m$ là số chẵn$\}$.*

Trường hợp $n = 0, m = 0$

Trường hợp n chẵn, $m = 0$

Trường hợp n lẻ, $m = 1$

Trường hợp $n = 0, m = 2$

Trường hợp $n = 0$, m chẵn

Kết hợp

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Xây dựng

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

2. Ý nghĩa mối liên hệ giữa văn phạm chính quy và FA

a) Sự tương đương hoàn toàn (Mathematical equivalence)

Ý nghĩa lớn nhất là tính tương đương: Một ngôn ngữ là chính quy nếu và chỉ nếu nó có thể được mô tả bằng Văn phạm chính quy và được chấp nhận bởi một FA.

- *Văn phạm chính quy (RG): Đóng vai trò là bộ quy tắc "sinh". Nó giống như công thức nấu ăn, hướng dẫn bạn cách lắp ghép các ký tự để tạo thành một chuỗi đúng.*
- *Automata hữu hạn (FA): Đóng vai trò là bộ máy "kiểm chứng". Nó giống như một nhân viên kiểm định, đứng ở cuối dây chuyền để xem sản phẩm có đúng công thức không.*

b) Chuyển đổi từ Tư duy sang Thuật toán

Con người thường tư duy theo quy tắc (văn phạm), nhưng máy tính cần các bước chuyển trạng thái cụ thể (Automata). Mối liên hệ này cho phép chúng ta:

- *Thiết kế quy tắc bằng ngôn ngữ tự nhiên/văn phạm.*
- *Tự động hóa việc chuyển quy tắc đó thành sơ đồ trạng thái.*
- *Cài đặt sơ đồ đó vào phần cứng hoặc phần mềm.*

3. Ứng dụng thực tế

a) Trình phân tích từ vựng (Lexical Analyzer/Scanner)

Trong các trình biên dịch như GCC hay Clang, mối liên hệ này được sử dụng triệt để:

- Văn phạm: Định nghĩa thế nào là một "số nguyên", "tên biến" hay "từ khóa".
- FA: Khi bạn nhấn "Compile", trình biên dịch sẽ chạy một FA được xây dựng từ văn phạm đó để quét qua mã nguồn của bạn. Nếu FA rơi vào trạng thái lỗi, bạn sẽ nhận được thông báo "Syntax Error".

b) Chế tạo công cụ sinh mã tự động (Scanner Generators)

Các công cụ nổi tiếng như Lex hay Flex hoạt động dựa trên mối liên hệ này.

- Lập trình viên chỉ cần viết các văn phạm chính quy (dưới dạng Regex).
- Công cụ sẽ tự động áp dụng các định lý toán học để xuất ra mã nguồn C/C++ chứa một DFA tối ưu dùng để nhận diện các mẫu đó.

c) Xử lý ngôn ngữ tự nhiên (NLP) ở mức cơ bản

Trong các hệ thống phân tích văn bản đơn giản:

- Người ta dùng văn phạm chính quy để mô tả cấu trúc câu hoặc từ vựng.
- FA được dùng để trích xuất thông tin (Information Extraction) với tốc độ cực nhanh mà không cần đến các mô hình AI phức tạp tốn tài nguyên.

d) Kiểm chứng và Đặc tả hệ thống

Trong kỹ thuật phần mềm, các yêu cầu hệ thống thường được viết dưới dạng quy tắc (Grammar). Việc chuyển chúng sang FA giúp các kỹ sư chạy mô phỏng để xem các quy tắc đó có mâu thuẫn nhau không, hoặc có dẫn đến "trạng thái chết" (Deadlock) nào không trước khi bắt đầu lập trình.

4. Tại sao mối liên hệ giữa văn phạm chính quy và FA quan trọng

Nếu không có mối liên hệ này, việc viết các chương trình kiểm tra chuỗi sẽ cực kỳ thủ công và dễ sai sót. Thay vì viết hàng trăm câu lệnh $if - else$ lồng nhau:

- Định nghĩa quy tắc bằng văn phạm.
- Hệ thống (thư viện ngôn ngữ như re trong Python, $java.util.regex$ trong Java) sẽ tự động xây dựng một FA ngầm định bên dưới.
- Kết quả là code của ngắn gọn hơn, dễ bảo trì hơn và hiệu suất cực kỳ cao (độ phức tạp$O(n)$).

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

IV. Tính chất của ngôn ngữ chính quy

1. Automata hữu hạn không xuất phát lại

Định nghĩa 2.4.1.

Định lý 2.4.2.

Giả sử DFA $M = (\{q_0, q_1, \dots, q_n\}, \Sigma, \delta, q_0, F)$. Xây dựng FA $M' = (Q \cup \{q_{n+1}\}, \Sigma, \delta', q_0, F')$ như sau:

$$\delta'(q,a) = \begin{cases} \delta(q,a) & khi\ \delta(q,a) \neq q_0 \\ q_{n+1} & \delta(q,a) = q_0 \end{cases} \quad với\ q \in Q$$

$$\delta(q_{n+1}, a) = \delta(q_0, a)$$

$$F' = \begin{cases} F & nếu\ q_0 \notin F \\ F \cup \{q_{n+1}\} & nếu\ q_0 \in F \end{cases}$$

M' thực hiện các bước chuyển hoàn toàn giống M trừ khi M chuyển về trạng thái q_0 thì M' chuyển về trạng thái q_{n+1}.

Ví dụ 2.4.3. *Xây dựng FA M không xuất phát lại đoán nhận các xâu trên bảng chữ $\{a, b\}$ có hậu tố bab.*

Bước 1.

Bước 2.

δ_2	a	b

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Do

Vậy

Bước 3.

M

2. Các phép toán trên ngôn ngữ chính quy

Định lý 2.4.4.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Giả sử L_1, L_2 là hai ngôn ngữ chính quy được đoán nhận bởi các DFA không xuất phát lại $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ và $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ với $Q_1 \cap Q_2 = \emptyset$. Xây dựng FA $M = (Q, \Sigma, \delta, q_0, F)$ đoán nhận $L_1 \cup L_2$ như sau:

$$Q = Q_1 \cup Q_2 \cup \{q_0\} \setminus \{q_{01}, q_{02}\}$$

$$F = \begin{cases} F_1 \cup F_2 \cup \{q_0\} \setminus \{q_{01}, q_{02}\} & \text{nếu } q_{01} \in F_1 \text{ hoặc } q_{02} \in F_2 \\ F_1 \cup F_2 & \text{nếu } q_{01} \notin F_1 \text{ hoặc } q_{02} \notin F_2 \end{cases}$$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & \text{nếu } q \in Q_1 \setminus \{q_{01}\} \\ \delta_2(q, a) & \text{nếu } q \in Q_2 \setminus \{q_{02}\} \end{cases}$$

$$\delta(q_0, a) = \delta_1(q_{01}, a) \cup \delta_2(q_{02}, a)$$

Do $Q_1 \cap Q_2 = \emptyset$ và M_1, M_2 là các DFA không xuất phát lại nên mỗi khi ở bước thứ nhất trong M thực hiện thì M sẽ bắt chước bước chuyển trạng thái đầu tiên hoặc của M_1 hoặc của M_2 rồi sau đó thực thi giống như M_1 hoặc M_2. Vậy $L(M) = L_1 \cup L_2$.

Ví dụ 2.4.5. *Xây dựng FA M đoán nhận các xâu nhị phân hoặc có tiền tố 11 hoặc có hậu tố 11.*

Bước 1.

Bước 2.

Bước 3.

δ_3	a	b

Do

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Bước 4.

Bước 5.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định lý 2.4.6. Nếu L là ngôn ngữ chính quy thì $\Sigma^* \setminus L$ cũng là ngôn ngữ chính quy.

Giả sử L được đoán nhận bởi $M = (Q, \Sigma, \delta, q_0, F)$. Xây dựng $M' = (Q, \Sigma, \delta, q_0, F')$ với $F' = Q \backslash F$.

Khi đó M' hoạt động giống hệt như M, chỉ khác do $F' = Q \backslash F$ nên các xâu M đoán nhận thì M' không đoán nhận còn các xâu M không đoán nhận thì M' đoán nhận.

Ví dụ 2.4.7. *Xây dựng DFA M đoán nhận các xâu trên bảng chữ $\{a, b\}$ không có hậu tố là $babb$.*

Bước 1.

Bước 2.

Định lý 2.4.8. Nếu L_1, L_2 là các ngôn ngữ chính quy trên bảng chữ Σ_1, Σ_2 thì $L_1 \cap L_2$ cũng là ngôn ngữ chính quy.

Theo công thức De–Morgan: $L_1 \cap L_2 = \overline{\overline{L_1 \cap L_2}} = \overline{\overline{L_1} \cup \overline{L_2}} = \Sigma^* \backslash \left((\Sigma^* \backslash L_1) \cup (\Sigma^* \backslash L_2) \right)$ nên $L_1 \cap L_2$ là ngôn ngữ chính quy.

Hệ quả 2.4.9. Lớp các ngôn ngữ chính quy tạo thành một đại số.

Hệ quả 2.4.10. Nếu L là một ngôn ngữ chính quy trên bảng chữ Σ và $\Sigma \subset \Sigma'$. Khi đó có một FA với bảng chữ vào Σ đoán nhận L khi và chỉ khi có một FA với bảng chữ vào Σ' đoán nhận L.

Định lý 2.4.11.

Giả sử L_1, L_2 là hai ngôn ngữ chính quy được đoán nhận bởi các DFA $M_1 = (Q_1, \Sigma_1, \delta_1, q_{01}, F_1)$ và $M_2 = (Q_2, \Sigma_2, \delta_2, q_{02}, F_2)$ với $Q_1 \cap Q_2 = \emptyset$.

Xây dựng FA $M = (Q, \Sigma, \delta, q_0, F)$ đoán nhận $L_1 L_2$ như sau:

$$Q = Q_1 \cup Q_2$$

$$\Sigma = \Sigma_1 \cup \Sigma_2$$

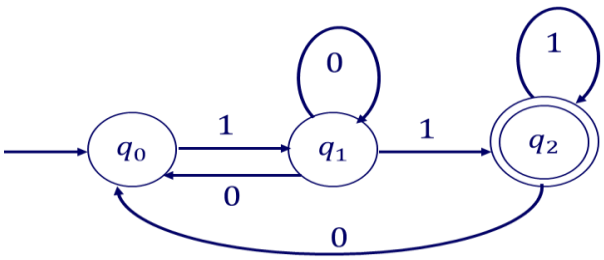
GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

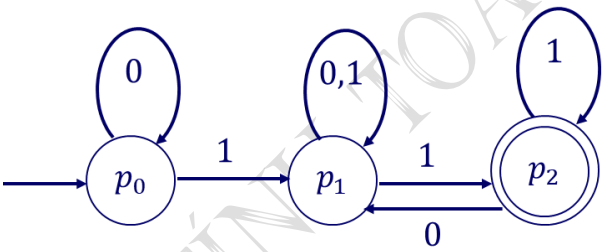
$$\delta(q,a)=\begin{cases}\{\delta_1(q,a)\} & \text{nếu } q\in Q_1\backslash F_1, a\in\Sigma_1\\ \{\delta_1(q,a),\delta_2(q_{02},a)\} & \text{nếu } q\in F_1, a\in\Sigma_1\\ \{\delta_2(q,a)\} & \text{nếu } q\in Q_2, a\in\Sigma_2\end{cases}$$

$$F=\begin{cases}F_2 & \text{nếu } \lambda\notin L_2\\ F_1\cup F_2 & \text{nếu } \lambda\in L_2\end{cases}$$

Ở phần tiền tố, M hoạt động giống hệt như M_1. Sau khi đến trạng thái kết thúc của M_1 thì hoặc M tiếp tục bắt chước M_1 hoặc chuyển sang bắt chước M_2, do đó M đoán nhận ngôn ngữ L_1L_2 nên L_1L_2 là ngôn ngữ chính quy.

Ví dụ 2.4.12. *Xây dựng DFA M đoán nhận ngôn ngữ L_1L_2 cho bởi các FA M_1, M_2 sau:*





Bước 1.

δ_3	0	1

Do

Vậy

Bước 2.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định lý 2.4.13.

Giả sử L được đoán nhận bởi DFA không xuất phát lại $M = (Q, \Sigma, \delta, q_0, F)$. Xây dựng FA $M' = (Q, \Sigma, \delta', q_0, F')$ đoán nhận L^* như sau:

$$\delta'(q,a) = \begin{cases} \{\delta(q,a)\} & \text{nếu } \delta(q,a) \notin Q\backslash F \\ \{\delta(q,a), q_0\} & \text{nếu } \delta(q,a) \in F \end{cases}$$

$$F' = F \cup \{q_0\}$$

M' luôn bắt chước M nhưng mỗi khi rơi vào trạng thái kết thúc thì hoặc M' cũng về trạng thái kết thúc hoặc quay về q_0. Do đó $L(M') = L^*$ nên L^* là ngôn ngữ chính quy.

Ví dụ 2.4.14*. Xây dựng FA M đoán nhận ngôn ngữ trên bảng chữ $\{a, b\}$ đoán nhận ngôn ngữ chính quy có biểu thức chính quy biểu diễn là $(bab + aba)^*$.*

Bước 1.

Bước 2.

Bước 3.

GIÁO TRÌNH MÔ HÌNH TOÁN

Bước 4.

V. Định lý Kleene

1. Định lý

Định lý Kleene, công bố năm 1956 bởi Stephen Kleene, được coi là định lý quan trọng nhất trong lý thuyết automata. Nó thiết lập một mối quan hệ tương đương tuyệt đối giữa các mô hình toán học khác nhau của ngôn ngữ chính quy.

Định lý 2.5.1. $\emptyset$ ***và*** $\{\lambda\}$ ***là các ngôn ngữ chính quy.***

Định lý 2.5.2. $\forall u \in \Sigma^*$ ***thì*** $\{u\}$ ***là ngôn ngữ chính quy.***

Hệ quả 2.5.3. Mọi tập con hữu hạn của Σ^* ***đều là ngôn ngữ chính quy.***

Hệ quả 2.5.4. Mọi ngôn ngữ tạo nên từ ngôn ngữ hữu hạn bằng cách áp dụng một số hữu hạn lần các phép hợp, ghép tiếp, bao đóng đều là ngôn ngữ chính quy.

Định lý 2.5.5 (Định lý Kleene). Mọi ngôn ngữ chính quy đều có thể nhận được từ các ngôn ngữ hữu hạn bằng cách áp dụng một số hữu hạn lần các phép hợp, ghép tiếp, bao đóng.

2. Ý nghĩa của định lý Kleene

Ý nghĩa lớn nhất của định lý này là khả năng hợp nhất các cách tiếp cận khác nhau để mô tả cùng một vấn đề:

- *Tính tương đương: Một ngôn ngữ là chính quy nếu và chỉ nếu nó có thể được mô tả bằng một biểu thức chính quy, hoặc được chấp nhận bởi một DFA, hoặc được chấp nhận bởi một NFA.*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- *Sự tự do trong biểu diễn: Định lý cho phép chúng ta chọn công cụ phù hợp nhất cho từng giai đoạn. Con người có thể dùng Regex để mô tả (vì nó ngắn gọn), nhưng máy tính sẽ dùng DFA để thực thi (vì nó nhanh).*
- *Tính đóng: Định lý Kleene giúp chứng minh các tính chất quan trọng của ngôn ngữ chính quy (như phép hợp, phép giao, phép lấy bù) một cách dễ dàng thông qua việc thao tác trên sơ đồ máy tự động.*

3. Hai chiều của định lý Kleene

Định lý này gồm hai phần chứng minh tương ứng với hai chiều chuyển đổi:

a) Chiều thuận: Từ biểu thức chính quy (RE) sang Automata (FA)

Chiều này chứng minh rằng mọi quy tắc mô tả bằng Regex đều có thể "chế tạo" thành một cái máy để chạy.

- Thuật toán tiêu biểu: Thuật toán Thompson.
- Ý nghĩa: Đây là cơ sở để các phần mềm (như trình duyệt, IDE) hiểu được các chuỗi Regex bạn viết.

b) Chiều nghịch: Từ Automata (FA) sang biểu thức chính quy (RE)

Chiều này chứng minh rằng bất kỳ hệ thống máy móc phức tạp nào (có trạng thái hữu hạn) đều có thể được tóm tắt lại bằng một dòng công thức Regex duy nhất.

- Thuật toán tiêu biểu: Phương pháp loại trừ trạng thái hoặc thuật toán Arden.
- Ý nghĩa: Giúp con người đọc hiểu được logic của một hệ thống máy móc hoặc một mạch số phức tạp.

4. Ứng dụng thực tế

a) Trình tối ưu hóa Regex

Các bộ máy xử lý Regex hiện đại (như trong thư viện re của Python hay grep của Linux) sử dụng định lý Kleene để:

- Nhận một Regex từ người dùng.
- Chuyển nó sang NFA (theo chiều thuận).
- Chuyển NFA đó sang DFA để tìm kiếm với tốc độ cực nhanh.

b) Thiết kế và kiểm chứng phần cứng (Hardware Verification)

Khi thiết kế một con chip (là một hệ thống các trạng thái hữu hạn), các kỹ sư dùng định lý Kleene để trích xuất ra biểu thức chính quy tương đương với hành vi của chip đó. Điều này giúp họ kiểm tra xem con chip có hoạt động đúng theo đặc tả ban đầu hay không.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

c) An ninh mạng và lọc dữ liệu

Các hệ thống phát hiện xâm nhập (IDS) mô tả các dấu hiệu tấn công bằng Regex. Nhờ định lý Kleene, các dấu hiệu này được nén lại thành các máy DFA cực kỳ tối ưu, cho phép quét hàng triệu gói tin mỗi giây mà không làm chậm mạng.

d) Lập trình trình biên dịch

Định lý Kleene là nền tảng cho các công cụ như Lex hoặc Flex. Nó giúp tự động hóa quá trình biến các quy tắc ngữ pháp của ngôn ngữ lập trình (như C, Java) thành các bộ máy quét từ vựng chạy trực tiếp trên máy tính.

Tóm tắt mối quan hệ giữa biểu thức chính quy, NFA, DFA:

Mô hình	Đặc điểm	Vai trò
Biểu thức chính quy	Đại số, khai báo	Mô tả (con người sử dụng)
NFA	Đồ thị, không đơn định	Thiết kế trung gian
DFA	Cấu trúc dữ liệu, đơn định	Thực thi (máy tính dùng)

Định lý Kleene chính là "chất keo" dính 3 mô hình này lại với nhau thành một hệ thống thống nhất.

BÀI TẬP BUỔI 4

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Nếu bạn không muốn cố gắng thì người khác muốn kéo bạn! lên cũng không biết tay bạn ở đâu!

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

CHAPTER III. AUTOMATA ĐẨY XUỐNG VÀ NGÔN NGỮ PHI NGỮ CẢNH

§1. VĂN PHẠM PHI NGỮ CẢNH VÀ CÂY SUY DẪN

I. Xuất xứ của văn phạm phi ngữ cảnh

1. Nguồn gốc ra đời

Xuất xứ của văn phạm phi ngữ cảnh (Context-Free Grammar - CFG) là mô tả ngôn ngữ tự nhiên. Văn phạm phi ngữ cảnh được Noam Chomsky giới thiệu năm 1956 – 1959 trong quá trình nghiên cứu cấu trúc hình thức của ngôn ngữ tự nhiên. Mục tiêu của Chomsky là xây dựng các mô hình toán học để mô tả cách con người tạo ra và hiểu câu trong ngôn ngữ tự nhiên, đặc biệt là các cấu trúc lồng nhau (ví dụ: mệnh đề phụ, ngoặc, quan hệ chủ ngữ –vị ngữ).

Một câu sinh ra qua các bước khai triển các cú pháp liên kết từ thành câu. Các nhà ngôn ngữ đặt hy vọng vào mô hình văn phạm phi ngữ cảnh để biểu diễn ngôn ngữ nhưng họ sớm nhận thấy mô hình này quá nhỏ bé, không đủ diễn tả hết sự phong phú và linh hoạt của ngôn ngữ tự nhiên.

Đặc trưng cốt lõi của văn phạm phi ngữ cảnh là mỗi sản xuất của văn phạm đều có dạng $A \longrightarrow \alpha$. Do có thể thay thế A bằng α ở bất kỳ đâu, bất kể những ký tự xung quanh A là gì. Nó không phụ thuộc vào "ngữ cảnh" nên văn phạm này được gọi là phi ngữ cảnh.

2. Sự giao thoa với khoa học máy tính

Gần như cùng thời điểm đó, các nhà khoa học máy tính đang đau đầu tìm cách định nghĩa cú pháp cho các ngôn ngữ lập trình đời đầu.

- *John Backus*: Khi thiết kế ngôn ngữ *FORTRAN*, ông đã bắt đầu suy nghĩ về cách mô tả cú pháp.
- *Peter Naur*: Năm 1959, khi soạn thảo báo cáo cho ngôn ngữ *ALGOL 60*, ông đã cải tiến cách ký hiệu của Backus.

Sự thật thú vị: Cách ký hiệu này được gọi là BNF (Backus-Naur Form). Sau đó, người ta mới nhận ra rằng BNF về mặt toán học hoàn toàn tương đương với văn phạm phi ngữ cảnh mà Chomsky đã đề xuất. Dạng chuẩn Backus – Naur được dùng để diễn tả cú pháp của ngôn ngữ lập trình cấp cao chính là một biến thể của văn phạm phi ngữ cảnh, trong đó dấu $:=$ thay thế $\longrightarrow$. Một biểu thức đại số có thể định nghĩa:

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 1.1.1. *Cho biểu thức* $(7-3)*(3+2)$.

a) *Cho văn phạm với tập sản xuất trên sinh ra biểu thức trên.*

b) *Vẽ cây suy dẫn sinh ra biểu thức trên.*

c) *Vẽ 2 cây suy diễn khác nhau cho biểu thức* $5+5*5$. *Từ đó suy ra văn phạm với tập sản xuất trên là văn phạm nhập nhằng.*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Do

Việc nghiên cứu văn phạm phi ngữ cảnh tạo nên một cơ sở lý luận cho việc biểu diễn ngôn ngữ lập trình, cho việc tìm kiếm các giải thuật phân tích cú pháp vận dụng trong các chương trình dịch và cho nhiều ứng dụng khác về xử lý các xâu.

3. Ý nghĩa và ứng dụng của văn phạm phi ngữ cảnh

Sự xuất hiện của CFG đã thay đổi hoàn toàn cách chúng ta xây dựng phần mềm:

- Trình biên dịch (Compilers): Hầu hết cú pháp của các ngôn ngữ lập trình hiện đại (C, Java, Python) được định nghĩa bằng CFG.
- Trình phân tích cú pháp (Parser): Các công cụ như Yacc, Bison hay Antlr sử dụng CFG để biến mã nguồn thành cây cú pháp (Parse Tree).
- Xử lý ngôn ngữ tự nhiên (NLP): Dùng để phân tích cấu trúc ngữ pháp của câu trước khi đưa vào các mô hình học sâu.

Văn phạm phi ngữ cảnh ra đời từ nỗ lực của **Chomsky** nhằm hiểu ngôn ngữ loài người, nhưng lại trở thành "ngôn ngữ mẹ" để chúng ta giao tiếp với máy tính thông qua các trình biên dịch.

II. Tính chất đóng của văn phạm phi ngữ cảnh

Định lý 1.2.1. Cho L_1, L_2 là hai ngôn ngữ phi ngữ cảnh cùng trên bảng chữ Σ. Khi đó:

- ***$L_1 \cup L_2, L_1 L_2$ là ngôn ngữ ngôn ngữ phi ngữ cảnh.***
- ***L_1^R, L_1^* là ngôn ngữ phi ngữ cảnh.***

Ví dụ 1.2.2. Cho ngôn ngữ $L = \{0^n 1^n 0^m | m, n \geq 1\}$.

a) L là ghép tiếp của hai ngôn ngữ nào? Xây dựng văn phạm sinh ra hai ngôn ngữ này.

L

Văn phạm

Văn phạm

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

b) L thuộc lớp ngôn ngữ nào?

G_1

G_2

L

Định lý 1.2.3. Lớp các ngôn ngữ phi ngữ cảnh không đóng với phép giao và phép lấy phần bù.

Ví dụ 1.2.4. *Cho ngôn ngữ* $L_1 = \{0^n 1^n 2^m | m, n \geq 0\}$, $L_2 = \{0^m 1^n 2^n | m, n \geq 0\}$.

a) *Chứng minh* L_1, L_2 *là các ngôn ngữ phi ngữ cảnh.*

Văn phạm

- Nếu
- Nếu
- Nếu
- Nếu

Vậy

Văn phạm

- Nếu
- Nếu
- Nếu
- Nếu

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Vậy

b) $L_1 \cap L_2$ có là ngôn ngữ phi ngữ cảnh không? Giải thích.

Ứng dụng: Tính chất này thường được dùng để chứng minh một ngôn ngữ không phải là phi ngữ cảnh bằng phương pháp phản chứng. Nếu giao một ngôn ngữ "nghi vấn" với một ngôn ngữ chính quy mà kết quả ra một ngôn ngữ không phải ngôn ngữ phi ngữ cảnh, thì ngôn ngữ nghi vấn đó chắc chắn không phải ngôn ngữ phi ngữ cảnh.

Định lý 1.2.5. Nếu L_1 là ngôn ngữ phi ngữ cảnh, L_2 là ngôn ngữ chính quy thì $L_1 \cap L_2$ là ngôn ngữ phi ngữ cảnh.

***Ví dụ 1.2.6**. Cho ngôn ngữ $L_1 = \{0^n 1^n 2^m | m, n \geq 0\}$, $L_2 = \{0^m 1^n 2^k | m, n, k \geq 0\}$.*

a) L_1, L_2 là các ngôn ngữ gì? Giải thích.

L_1 là ngôn ngữ phi ngữ cảnh (theo ví dụ 1.2.4).

L_1

Văn phạm

Văn phạm

Văn phạm

GIÁO TRÌNH MÔ HÌNH TOÁN

b) $L_1 \cap L_2$ có là ngôn ngữ phi ngữ cảnh không? Giải thích.

Định lý 1.2.7. Nếu thay thế mỗi ký tự trong ngôn ngữ phi ngữ cảnh bằng một ngôn ngữ phi ngữ cảnh khác thì ngôn ngữ nhận được vẫn là ngôn ngữ phi ngữ cảnh.

III. Cây suy dẫn trong văn phạm phi ngữ cảnh

1. Cây suy dẫn

Trong văn phạm phi ngữ cảnh, ***cây suy dẫn*** (còn gọi là *cây phân tích cú pháp*, *parse tree*) là một biểu diễn dạng cây cho thấy cách một xâu được sinh ra từ ký hiệu bắt đầu thông qua việc áp dụng các luật sinh của văn phạm. Cây suy dẫn phản ánh rõ ràng cấu trúc phân cấp của xâu, vượt trội so với cách biểu diễn tuyến tính bằng suy dẫn.

Định nghĩa 1.3.1.

-
-
-
-
-
-

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định lý 1.3.2. Cho $G = (V, \Sigma, S, P)$ là một văn phạm phi ngữ cảnh và $u \in \Sigma^*$. Khi đó $u \in L(G)$ khi và chỉ khi tồn tại một cây suy dẫn đầy đủ trong G có kết quả là u.

2. Suy dẫn trái và suy dẫn phải

Định nghĩa 1.3.3.

-
-

Ví dụ 1.3.4. *Cho văn phạm $G = (\{S, A, B\}, \{0,1\}, S, \{S \longrightarrow 0S|AB, A \longrightarrow A0|1B, B \longrightarrow B1|1\})$ và xâu $u = 000111000111$.*

a) *Tìm suy dẫn trái nhất của xâu u.*

b) *Tìm suy dẫn phải nhất của xâu u.*

c) *Vẽ cây suy dẫn của xâu u.*

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định lý 1.3.5. Nếu $S \Longrightarrow^* u$ là một suy dẫn trong văn phạm phi ngữ cảnh G thì sẽ tồn tại một suy dẫn trái nhất của xâu u.

Hệ quả 1.3.6. Mọi cây suy dẫn của xâu u đều tạo ra suy dẫn trái nhất của u.

3. Sự nhập nhằng của văn phạm phi ngữ cảnh

Định nghĩa 1.3.7.

Văn phạm nhập nhằng rất nguy hiểm khi lập trình vì máy tính hiểu một biểu thức theo hai cách khác nhau, kết quả sẽ sai lệch.

4. Ý nghĩa và ứng dụng của cây suy dẫn trong khoa học máy tính

Cây suy dẫn giữ vai trò trung tâm trong nhiều lĩnh vực:

- *Trong trình biên dịch*: Cây suy dẫn là bước đệm để máy tính hiểu cấu trúc câu lệnh. Từ cây này, trình biên dịch sẽ tạo ra cây cú pháp trừu tượng để bắt đầu tính toán hoặc tạo mã máy.
- *Xử lý ngôn ngữ tự nhiên*: Giúp phân tích cấu trúc ngữ pháp của một câu nói (đâu là chủ ngữ, vị ngữ, cụm danh từ).
- *Tính toán logic*: Xác định độ ưu tiên của các toán tử trong toán học.
- *Lý thuyết ngôn ngữ hình thức*: chứng minh các tính chất của ngôn ngữ phi ngữ cảnh.
- *Ngôn ngữ lập trình*: xác định cấu trúc cú pháp hợp lệ của chương trình.

Cây suy dẫn biến những sản xuất $A \longrightarrow \alpha$ thành một bản đồ trực quan, giúp chúng ta kiểm soát cấu trúc và độ chính xác của ngôn ngữ.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

IV. Rút gọn văn phạm phi ngữ cảnh

1. Mục đích của việc rút gọn văn phạm phi ngữ cảnh

Trong lý thuyết ngôn ngữ hình thức, một văn phạm phi ngữ cảnh có thể chứa các thành phần không cần thiết nhưng không làm thay đổi ngôn ngữ được sinh ra. Việc rút gọn văn phạm phi ngữ cảnh nhằm loại bỏ các thành phần dư thừa đó, từ đó:

- Làm cho văn phạm đơn giản, gọn gàng và dễ phân tích hơn.
- Thuận lợi cho việc chứng minh lý thuyết.
- Là bước chuẩn bị quan trọng cho các phép chuẩn hóa văn phạm.
- Hỗ trợ xây dựng PDA và trình biên dịch.

Điều quan trọng khi rút gọn văn phạm phi ngữ cảnh là: văn phạm sau khi rút gọn phải sinh ra cùng một ngôn ngữ như văn phạm ban đầu.

2. Ký hiệu vô ích

Định nghĩa 1.4.1.

-

Nhận xét. X là ký hiệu vô ích nếu:

- ***Từ một xâu chứa X không thể suy dẫn ra bất cứ xâu $u \in \Sigma^*$. Ký hiệu X có tính chất này gọi là ký hiệu vô sinh.***
- ***Từ S không suy dẫn ra bất cứ xâu nào chứa X. Ký hiệu X có tính chất này gọi là ký hiệu không đến được.***

Ví dụ 1.4.2. Cho văn phạm phi ngữ cảnh $G = (\{S, A, B\}, \{a, b, c\}, S, \{S \longrightarrow aSb|aA, A \longrightarrow bA|a, B \longrightarrow cB\})$.

-

-

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Bổ đề 1.4.3 (Loại bỏ ký hiệu vô sinh). Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$ với $L(G) \neq \emptyset$. Khi đó luôn tồn tại văn phạm phi ngữ cảnh $G' = (V, \Sigma', S, P')$ không chứa ký hiệu vô sinh và tương đương với G.

Từ

- Nếu
- Nếu

Lặp

Bổ đề 1.4.4 (Loại bỏ ký hiệu không đến được). Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$. Khi đó luôn tồn tại văn phạm phi ngữ cảnh $G' = (V', \Sigma', S, P')$ không chứa ký hiệu không đến được và tương đương với G.

Từ

- Đưa
- Nếu

Tiếp tục

Định lý 1.4.5. Mọi ngôn ngữ phi ngữ cảnh khác $\emptyset$ đều có thể sinh từ ngôn ngữ phi ngữ cảnh không chứa ký hiệu vô ích.

Giả sử $L \neq \emptyset$ là ngôn ngữ phi ngữ cảnh được sinh bởi văn phạm phi ngữ cảnh G. Áp dụng lần lượt bổ đề 1.4.3 và 1.4.4 sẽ được văn phạm phi ngữ cảnh G'' tương đương với G sinh ra L và không chứa ký hiệu vô ích.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 1.4.6. *Cho văn phạm phi ngữ cảnh* $G = (\{a, b, c\}, \{S, A, B, C, D, E\}, S, \{S \longrightarrow Ab|aC|BD, A \longrightarrow bC, B \longrightarrow a, C \longrightarrow ab,\ D \longrightarrow aD|Db, E \longrightarrow ac\})$.

a) Loại bỏ các ký hiệu vô ích trong G.

Bước 1.

-

-

-

-

-

Vậy

Bước 2.

-

-

Vậy

G_2

b) Áp dụng bổ đề 1.4.4 rồi đến bổ đề 1.4.3 cho văn phạm G.

Bước 1.

-

-

Vậy

Bước 2.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

-

-

-

Vậy

Nhận xét. $G_4 \neq G_2$ *và trong* G_4 *vẫn còn ký hiệu vô ích.*

Chú ý. Áp dụng bổ đề 1.4.3 rồi mới áp dụng bổ đề 1.4.4.

3. $\lambda-$ sản xuất

Định nghĩa 1.4.7.

Định nghĩa 1.4.8.

•

•

Bổ đề 1.4.9 (loại bỏ $\lambda-$ ***sản xuất). Cho văn phạm phi ngữ cảnh*** $G = (V, \Sigma, S, P)$***. Khi đó tồn tại văn phạm phi ngữ cảnh*** $G' = (V, \Sigma, S, P')$ ***không chứa*** $\lambda-$ ***sản xuất và*** $L(G') = L(G)\backslash\{\lambda\}$***.***

Xây dựng tập sản xuất P' theo quy tắc sau:

- Nếu

+

+

+

- Lặp

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Ví dụ 1.4.10. *Cho văn phạm* $G = (\{0,1\}, \{S, A, B, C\}, S, \{S \longrightarrow A0B|0A|1, A \longrightarrow BC|1C0, B \longrightarrow C|01, C \longrightarrow \lambda\})$.

a) Xây dựng văn phạm phi ngữ cảnh G_1 *không chứa* λ *– sản xuất và* $L(G_1) = L(G)\backslash\{\lambda\}$.

Bước 1.

-

-

-

Bước 2.

-

-

-

-

- Giữ

Vậy

b) Xây dựng văn phạm phi ngữ cảnh G_2 *không chứa sản xuất đơn và tương đương với* G_1.

Bước 3.

- Đưa

-

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

c) *Xây dựng văn phạm phi ngữ cảnh G' không chứa ký hiệu vô ích tương đương với G.*

Bước 4.

- Do
- Do
- Do

Vậy

Bước 5.

- Đưa
- Do

Vậy

G_4

4. Các sản xuất đơn

Định nghĩa 1.4.11.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Bổ đề 1.4.12

Xây dựng tập sản xuất P' theo quy tắc sau:

- Đưa
- Nếu
- Lặp

Định lý 1.4.13.

Áp dụng lần lượt các bổ đề 1.4.9, 1.4.12, 1.4.3, 1.4.4 xây dựng được văn phạm G'.

Ví dụ 1.4.14. *Cho văn phạm phi ngữ cảnh* $G = (\{0,1,2\}, \{S, A, B, C, D, E\}, S, \{S \longrightarrow A0B|B1|E2,\ A \longrightarrow BD|01, B \longrightarrow DD|BD|B1,\ C \longrightarrow C2|12,\ D \longrightarrow \lambda|1, E \longrightarrow EE\})$. *Xây dựng văn phạm phi ngữ cảnh* G' *không chứa ký hiệu vô ích, không chứa* λ *– sản xuất, không chứa sản xuất đơn mà* $L(G') = L(G)\backslash\{\lambda\}$.

Bước 1.

-
-
-

Bước 2.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

-

-

-

-

-

-

- Giữ

Vậy

Bước 3.

- Đưa

-

-

-

-

Vậy

Bước 4.

- Do

- Do

- Do

- Do

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Do

Vậy

Bước 5.

- Đưa
- Do
- Do
- Do

Vậy

G_4

§2. HAI DẠNG CHUẨN QUAN TRỌNG

I. Mục đích của việc chuẩn hóa văn phạm phi ngữ cảnh

Chuẩn hóa văn phạm nhằm đưa văn phạm về một dạng tiêu chuẩn với cấu trúc luật sinh đơn giản, thống nhất, nhưng không làm thay đổi ngôn ngữ sinh ra.

Việc đưa văn phạm phi ngữ cảnh về các chuẩn dạng có ý nghĩa đặc biệt trong:

- Chứng minh lý thuyết về ngôn ngữ phi ngữ cảnh.
- Thiết kế và phân tích thuật toán phân tích cú pháp.
- Chứng minh các định lý đóng.
- Xây dựng máy đẩy xuống (PDA) tương đương

Hai chuẩn dạng quan trọng nhất là:

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Chuẩn dạng Chomsky (Chomsky Normal Form – CNF).
- Chuẩn dạng Greibach (Greibach Normal Form – GNF).

II. Dạng chuẩn Chomsky

1. Định nghĩa

Định nghĩa 2.2.1.

Ví dụ 2.2.2.

➢ *Văn phạm*

➢ *Văn phạm*

Nhận xét. Văn phạm phi ngữ cảnh G ở dạng chuẩn Chomsky nếu:

- ***Không chứa λ – sản xuất.***
- ***Không chứa sản xuất đơn***
- ***Các sản xuất mà vế phải chứa cả ký hiệu kết thúc và không kết thúc.***
- ***Các sản xuất mà vế phải có nhiều hơn hai ký hiệu.***

2. Đưa văn phạm về dạng chuẩn Chomsky

Định lý 2.2.3. Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$ với $\lambda \notin L(G)$. Khi đó tồn tại văn phạm phi ngữ cảnh $G' = (V, \Sigma', S, P')$ tương đương với G và ở dạng chuẩn Chomsky.

Theo định lý 1.4.13, luôn có thể giả sử G không chứa ký hiệu vô ích, không chứa λ – sản xuất, không chứa sản xuất đơn.

Bước 1.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Bước 2.

- Nếu
- Nếu

Lặp lại

Bước 3.

- Thêm
- Bỏ đi

Sau

Sau

Ví dụ 2.2.4. *Đưa văn phạm phi ngữ cảnh* $G = (\{0,1\}, \{S, A, B, C\}, S, \{S \longrightarrow AB|0BC1|101, A \longrightarrow 0BC|011, B \longrightarrow C1B|0, C \longrightarrow 001|10\})$ *về dạng chuẩn Chomsky.*

Bước 1.

Thêm

- Thay
- Thay

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Thay

- Thay

- Thay

- Thay

- Thay

Vậy

Bước 2.

- Với

- Với

- Với

- Với

- Với

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Với

Vậy

Nhận xét. Trong bước 2, để giảm bớt các ký hiệu không kết thúc thêm vào quá nhiều, tiến hành như sau:

- Với

- Với

- Với

- Với

- Với

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Với

Vậy

Ví dụ 2.2.5. *Đưa văn phạm phi ngữ cảnh* $G = (\{a, b\}, \{S, A, B\}, S, \{S \longrightarrow ABAB|aAB|ABb|bab,\ A \longrightarrow ABa|bba, B \longrightarrow bAB|baa\})$ *về dạng chuẩn Chomsky.*

Bước 1.

Bước 2.

Cách 1. Với

- Với

- Với

- Với

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Với

- Với

- Với

- Với

- Với

- Với

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Cách 2. Với

- Với
- Với
- Với
- Với
- Với
- Với
- Với
- Với
- Với
- Với

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

3. Ý nghĩa và ứng dụng của dạng chuẩn Chomsky

- *Cấu trúc cây nhị phân*: Mọi cây suy dẫn trong CNF đều là cây nhị phân. điều này cực kỳ thuận lợi cho việc lập trình và lưu trữ dữ liệu.
- *Thuật toán CYK*: CNF là điều kiện bắt buộc để chạy thuật toán CYK (Cocke-Younger-Kasami) — một thuật toán dùng để kiểm tra xem một chuỗi có thuộc về một văn phạm hay không với độ phức tạp $O(n^3)$.
- *Giới hạn độ cao của cây*: Nếu chuỗi có độ dài n, cây suy dẫn trong CNF sẽ có đúng $2n-1$ đỉnh trong.

III. Dạng chuẩn Greibach

1. Định nghĩa

Định nghĩa 2.3.1.

Ví dụ 2.3.2.

Nếu

Nhận xét. Mọi suy dẫn đều bắt đầu bằng ký hiệu kết thúc.

- ***Mỗi bước suy dẫn sinh ra ít nhất một ký tự đầu vào.***
- ***GNF đặc biệt phù hợp với phân tích cú pháp từ trên xuống (top-down parsing).***

2. Đưa văn phạm về dạng chuẩn Chomsky

Định lý 2.3.3. Cho văn phạm phi ngữ cảnh $G=(V,\Sigma,S,P)$ với $\lambda \notin L(G)$. Khi đó tồn tại văn phạm phi ngữ cảnh $G'=(V,\Sigma',S,P')$ tương đương với G và ở dạng chuẩn Greibach.

Ví dụ 2.3.4. Cho văn phạm $G=(\{S,A,B\},\{0,1\},S,\{S\longrightarrow 0A1B0|01, A\longrightarrow 1B0|110, B\longrightarrow 01\})$. Đưa văn phạm G về dạng chuẩn Greibach.

- Thêm

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Thay

- Thay

- Thay

Vậy

3. Ý nghĩa và ứng dụng của dạng chuẩn Greibach

- Tính kết thúc: Vì mỗi bước suy dẫn đều tạo ra đúng một ký hiệu kết thúc ở đầu chuỗi, chúng ta luôn biết chắc chắn rằng quá trình sinh chuỗi đang tiến triển và không bị lặp vô tận.
- Máy tự động đẩy xuống (PDA): GNF có mối liên hệ trực tiếp với PDA. Mỗi sản xuất trong GNF tương ứng với một bước chuyển của PDA: đọc một ký tự a và đẩy các biến α vào ngăn xếp.
- Kiểm soát độ dài: Độ dài của suy dẫn cho một chuỗi có độ dài n sẽ đúng bằng n bước.

IV. So sánh CNF và GNF

1. Bảng so sánh CNF và GNF

Đặc điểm	Dạng chuẩn Chomsky (CNF)	Dạng chuẩn Greibach
Quy tắc cơ bản	$A \longrightarrow BC$ hoặc $A \longrightarrow a$	$A \longrightarrow a\alpha$
Ký hiệu bắt đầu	Không bắt buộc bắt đầu bằng ký hiệu kết thúc	Luôn bắt buộc bắt đầu bằng ký hiệu kết thúc
Số ký hiệu vế phải	2 ký hiệu không kết thúc hoặc 1 ký hiệu kết thúc	1 ký hiệu kết thúc với hậu tối là xâu ký hiệu không kết thúc
Có $\lambda -$ sản xuất	Không (trừ $S \longrightarrow \lambda$ nếu cần)	Không
Cây suy dẫn	Cây nhị phân cân đối	Cây lệch về một phía
Thuật toán liên quan	Thuật toán CYK	Automata đẩy xuống (PDA)
Ưu điểm	Dễ tính độ phức tạp	Luôn tạo ra ngay ký hiệu kết thúc
Độ khó chuyển từ CFG	Tương đối dễ (chia nhỏ sản xuất)	Khó hơn (cần khử đệ quy trái)

2. Tại sao cần cả CNF và GNF

Khi xây dựng một trình biên dịch:

- Dùng **CNF** khi muốn kiểm tra xem mã nguồn có đúng cú pháp hay không (nhờ thuật toán CYK rất nhanh).
- Dùng **GNF** khi muốn mô hình hóa cách bộ nhớ (ngăn xếp) hoạt động khi thực thi các câu lệnh lồng nhau.

Cả hai đều giúp loại bỏ sự mơ hồ và những vòng lặp vô tận mà các văn phạm phi ngữ cảnh thông thường hay gặp phải.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

§3. AUTOMATA ĐẨY XUỐNG

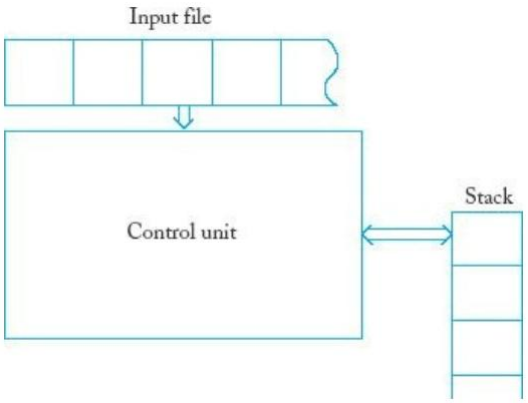
I. Mô tả automata đẩy xuống

Trong chương II ta đã thấy mỗi ngôn ngữ chính quy đều do một văn phạm chính quy sinh ra và đều có một DFA hoặc NFA đoán nhận tức là lớp ngôn ngữ do văn phạm chính quy sinh ra và lớp ngôn ngữ do FA đoán nhận là trùng nhau. Trong chương này ta sẽ thấy lớp ngôn ngữ do các văn phạm phi ngữ cảnh sinh ra trùng với lớp các ngôn ngữ được đoán nhận bởi automata đẩy xuống không đơn định (Nondeteministic Pushdown Automata – NPDA).

Chỉ có các NPDA mới đoán nhận được hết các ngôn ngữ do văn phạm phi ngữ cảnh sinh ra. Lớp các automata đẩy xuống đơn định (Deteministic Pushdown Automata – DPDA) chỉ đoán nhận được một tập con thực sự của lớp các ngôn ngữ phi ngữ cảnh. Tuy nhiên lớp các ngôn ngữ được đoán nhận bởi DPDA khá rộng, bao gồm phần lớn các ngôn ngữ lập trình.

Một PDA cũng giống FA có các thành phần:

- Một băng vào chứa xâu cần đoán nhận.
- Một bộ điều khiển là tập hữu hạn các trạng thái.
- Một đầu đọc cho phép đọc từ trái sang phải các ký hiệu trên băng vào.
- Khác biệt giữa PDA với FA là thêm một băng làm việc, gọi là ngăn xếp (stack), nhờ nó mà khả năng ghi nhớ của PDA được tăng thêm. Ngăn xếp được tổ chức theo nguyên tắc LIFO (Last in first out).



Căn cứ vào trạng thái hiện tại của bộ điều khiển, ký hiệu vào mà đầu đọc quan sát được và ký hiệu ở đầu ngăn xếp, PDA có hai phương án:

-

-

GIÁO TRÌNH MÔ HÌNH TOÁN

PDA có hai cách đoán nhận xâu như sau:

-

-

Ví dụ 3.1.1. *Cho ngôn ngữ* $L = \{ucu^R | u \in \{a\ b\}^*\}$

a) Mô tả ngôn ngữ L.

L là tập các xâu đối xứng trên bảng chữ $\{a, b, c\}$, có ký hiệu c làm tâm đối xứng, trong đó:

L

-

-

b) Xây dựng văn phạm G *sinh ra* L, *từ đó suy ra* L *thuộc lớp ngôn ngữ nào.*

Văn phạm

c) Xây dựng PDA đoán nhận ngôn ngữ L.

PDA

- Ở

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Ở

- Ở

Trường hợp 1.

Trường hợp 2.

-

-

d) Bằng định nghĩa, xây dựng PDA M đoán nhận ngôn ngữ L, từ đó đoán nhận các xâu $v_1 = aababcbabaa$ và $v_2 = aababcbabba$.

PDA

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Xâu

Xâu

d) Vẽ đồ thị chuyển mô tả M.

với

Nhận xét. Do ngăn xếp có khả năng lưu giữ các ký hiệu nên PDA có thể ghi nhớ nửa đầu của xâu v rồi sau đó so sánh nó với nửa cuối của xâu v. FA không có khả năng ghi nhớ này.

II. Automata đẩy xuống không đơn định

Định nghĩa 3.2.1 Một automata đẩy xuống không đơn định (NPDA) là một bộ bảy $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ trong đó:

- ***Q là tập hữu hạn các trạng thái bên trong của bộ điều khiển.***
- ***Σ là tập hữu hạn khác $\emptyset$ các ký hiệu, gọi là bảng chữ vào. Mỗi ký hiệu thuộc Σ gọi là ký hiệu vào.***
- ***Γ là tập hữu hạn khác $\emptyset$ các ký hiệu, gọi là bảng chữ ngăn xếp.***
- ***$q_0 \in Q$ là trạng thái đầu.***

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- $z_0 \in \Gamma$ ***là ký hiệu đặc biệt, gọi là ký hiệu bắt đầu ngăn xếp.***
- $F \subset Q$ ***gọi là tập các trạng thái kết thúc.***
- $\delta: Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \longrightarrow$ ***một tập con hữu hạn của*** $Q \times \Delta^*$***, được gọi là hàm chuyển trạng thái của*** M***.***

Định nghĩa 3.2.2.

Khi đó

Định nghĩa 3.2.3.

Khi đó

Ở mỗi thời điểm, tình huống tức thời của NPDA M được xác định bởi ba yếu tố sau:

- Xâu
- Trạng
- Phần

Định nghĩa 3.2.4.

Giả sử

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Nếu

- Nếu

Tương tự

Ví dụ 3.2.5. *Cho PDA* $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, z_0\}, \delta, q_0, z_0, \{q_2\})$ *với hàm chuyển trạng thái* δ *xác định như sau:* $\delta(q_0, a, z_0) = \{(q_0, a)\}, \delta(q_0, a, a) = \{(q_0, aa)\}, \delta(q_0, \lambda, a) = \{(q_1, a)\}, \delta(q_1, b, a) = \{(q_1, \lambda)\}, \delta(q_0, \lambda, z_0) = \delta(q_1, \lambda, \lambda) = \{(q_2, \lambda)\}$.

a) *Vẽ đồ thị chuyển mô tả* M.

b) *Cho* M *đoán nhận xâu* a^3b^3, *từ đó chứng minh* $L(M) = \{a^n b^n | n \geq 0\}$.

Xâu

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Xâu

Nếu

Nếu

Dãy

c) Xây dựng DPDA M_1 cũng đoán nhận ngôn ngữ $L(M)$.

Trong

Xây

Nếu

Nếu

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Dãy

III. Văn phạm phi ngữ cảnh và automata đẩy xuống

1. Ngôn ngữ được đoán nhận bởi automata đẩy xuống

Định nghĩa 3.3.1.

-
-
-

Ký hiệu

$T(M)$ được gọi là ngôn ngữ được đoán nhận bởi M theo tập trạng thái kết thúc.

Định nghĩa 3.3.2.

-
-
-

Ký hiệu

$N(M)$ được gọi là ngôn ngữ được đoán nhận bởi M theo ngăn xếp rỗng.

Ý nghĩa: Khi thiết kế PDA, ta có quyền chọn cách nào thuận tiện nhất cho bài toán của mình. Thông thường, khi chuyển đổi từ văn phạm phi ngữ cảnh (CFG) sang PDA, cách dùng ngăn xếp rỗng thường đơn giản và trực quan hơn.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

2. Mối liên hệ giữa văn phạm phi ngữ cảnh và automata đẩy xuống

Định lý 3.3.3. Cho ngôn ngữ phi ngữ cảnh L. Khi đó tồn tại một NPDA M đoán nhận L theo tập trạng thái kết thúc.

Giả sử

-

-

-

Hàm chuyển trạng thái δ được xây dựng như sau:

-

-

-

-

Khi đó

Chú ý.

- ✓ ***Khi ký hiệu ở đỉnh ngăn xếp là ký hiệu không kết thúc thì đầu đọc sẽ không đọc, PDA sẽ thực hiện bước chuyển nhẳm mắt.***
- ✓ ***Khi ký hiệu đầu đọc đọc được và ký hiệu ở đỉnh ngăn xếp trùng nhau thì đầu đọc dịch sang phải một ô, giữ nguyên trạng thái q_1, ký hiệu ở đỉnh ngăn xếp bị đẩy ra và không có ký hiệu được đẩy vào.***

Ví dụ 3.3.4. Cho văn phạm phi ngữ cảnh $G = (\{a, b, c\}, \{S, A, B\}, S, \{S \longrightarrow aSb|bAa, A \longrightarrow bAa|aBb, B \longrightarrow aBb|acb\})$.

a) Tìm $L(G)$.

Vậy

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

b) Xây dựng PDA M đoán nhận $L(G)$ theo tập trạng thái kết thúc và cho M đoán nhận xâu $u = aabaaacbbbabb$.

- Do
- Do
- Do

Vậy

Xâu

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

Định lý 3.3.5. Cho PDA M. Khi đó tồn tại một NPDA M' sao cho $N(M') = T(M)$.

Giả sử $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$. Xây dựng $M' = (Q', \Sigma', \Gamma', \delta', q_0', z_0', F')$ như sau:

- $Q' = Q \cup \{p_1, p_2\}$ với $p_1, p_2 \notin Q$.
- $q_0' = p_1$.
- $\Gamma' = \Gamma \cup \{\$\}$, $\$ \notin \Gamma$.
- $z_0 = \$$.
- $F' = \emptyset$.

Hàm chuyển trạng thái δ xác định như sau:

- $\delta'(p_1, \varepsilon, \$) = \{(q_0, \$z_0)\}$.
- $\delta'(q, a, z) = \delta(q, a, z) \ \forall q \in Q, a \in \Sigma, z \in \Gamma$.
- $\delta'(q, \lambda, z) = \begin{cases} \delta(q, \lambda, z) & khi\ q \in Q \backslash F, z \in \Gamma \\ \delta(q, \lambda, z) \cup \{(p_2, \lambda)} & khi\ q \in F,\ z \in \Gamma \end{cases}$.
- $\delta'(q, \lambda, \$) = \{(p_2, \lambda)\}$ nếu $q \in F$.
- $\delta'(p_2, \lambda, z) = \{(p_2, \lambda)\}$.

Khi đó có thể chứng minh được rằng $N(M') = T(M)$.

Định lý 3.3.6. Cho PDA M. Khi đó tồn tại một văn phạm phi ngữ cảnh G sao cho $L(G) = N(M)$.

Nhận xét.

- ***Theo định lý 3.3.3. lớp ngôn ngữ phi ngữ cảnh nằm trong lớp ngôn ngữ được đoán nhận bởi NPDA theo tập trạng thái kết thúc.***
- ***Theo định lý 3.3.5, lớp ngôn ngữ được đoán nhận bởi PDA theo tập trạng thái kết thúc nằm trong lớp ngôn ngữ được đoán nhận bởi NPDA theo ngăn xếp rỗng.***
- ***Theo định lý 3.3.6, lớp ngôn ngữ được đoán nhận bởi PDA theo tập ngăn xếp rỗng nằm trong lớp ngôn ngữ được sinh bởi văn phạm phi ngữ cảnh.***

Định lý 3.3.7. Một ngôn ngữ là phi ngữ cảnh khi và chỉ khi nó được một NPDA đoán nhận.

IV. Automata đẩy xuống đơn định và ngôn ngữ phi ngữ cảnh đơn định

Định nghĩa 3.4.1. Một automata đẩy xuống $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ được gọi là đơn định (DPDA) nếu nó được định nghĩa như trong định nghĩa 3.2.1 với các hạn chế sau:

- ***$\delta(q, a, z)$ chứa nhiều nhất một phần tử $\forall q \in Q, a \in \Sigma \cup \{\lambda\}, z \in \Gamma$.***
- ***Nếu $\delta(q, \lambda, z) \neq \emptyset$ với $q \in Q, z \in \Gamma$ thì $\delta(q, b, z) = \emptyset \ \forall b \in \Sigma$.***

Điều kiện đầu tiên yêu cầu đối với bất cứ trạng thái nào, đầu đọc đang đọc ký hiệu gì và giá trị của ngăn xếp là bao nhiêu thì chỉ có tối đa một bước chuyển. Điều kiện thứ hai là nếu có thể có

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

bước chuyển "nhắm mắt" ở trạng thái q, ký hiệu ngăn xếp z thì không thể có bước chuyển thông thường.

Định nghĩa 3.4.2. Ngôn ngữ L được gọi là ngôn ngữ phi ngữ cảnh đơn định khi và chỉ khi tồn tại một DPDA M thỏa mãn $L = L(M)$.

Chú ý.

- ✓ ***Trong FA: DFA và NFA có sức mạnh ngang nhau (đều đoán nhận ngôn ngữ chính quy).***
- ✓ ***Trong PDA: NPDA mạnh hơn DPDA.***
- ***NPDA: Đoán nhận toàn bộ lớp ngôn ngữ phi ngữ cảnh (CFL).***
- ***DPDA: Chỉ đoán nhận một tập con gọi là ngôn ngữ phi ngữ cảnh đơn định (DCFL).***

V. Văn phạm phi ngữ cảnh mô tả định dạng dữ liệu JSON và liên hệ CFG – PDA – JSON Parser

1. Giới thiệu chung

JSON (JavaScript Object Notation) là một định dạng dữ liệu dạng văn bản, nhẹ và độc lập nền tảng, được sử dụng rộng rãi trong các hệ thống phần mềm hiện đại như dịch vụ web (*REST API*), ứng dụng web, hệ thống phân tán và cơ sở dữ liệu *NoSQL*. *JSON* cho phép biểu diễn dữ liệu có cấu trúc phân cấp thông qua các đối tượng (object) và mảng (array), với khả năng lồng nhau ở mức bất kỳ.

Do mang đặc trưng cấu trúc lồng nhau và quan hệ thứ bậc, cú pháp của *JSON* không thể được mô tả đầy đủ bằng văn phạm chính quy, mà cần sử dụng văn phạm phi ngữ cảnh. Đây là một ví dụ thực tế tiêu biểu cho vai trò của văn phạm phi ngữ cảnh trong khoa học máy tính hiện đại, bên cạnh các ngôn ngữ lập trình.

2. Các thành phần cú pháp của JSON

Về mặt cú pháp, một tài liệu *JSON* được xây dựng từ các thành phần cơ bản sau:

- *Giá trị* (*value*):

- *Đối tượng* (*object*):

- *Mảng* (*array*):

- *Chuỗi* (*string*):

Mỗi tài liệu *JSON* hợp lệ luôn tương ứng với đúng một cây cú pháp, phản ánh cấu trúc phân cấp của dữ liệu.

GIÁO TRÌNH MÔ HÌNH TOÁN

3. Văn phạm phi ngữ cảnh mô tả JSON (dạng chuẩn hóa)

Bảng chữ

Ký hiệu

Object :

Members :

Pair :

Value :

Array :

Elements :

Vì sao đây là CFG?

- Có đệ quy lồng nhau (*object* trong *object*, *array* trong *array*)
- Không cần thông tin ngữ cảnh
- Mỗi sản xuất có dạng $A \to \alpha$.

JSON không phải ngôn ngữ chính quy, vì:

- Không thể kiểm tra ngoặc {} và [] lồng nhau bằng FA.
- Cần ngăn xếp nên là PDA.

4. Tại sao JSON cần văn phạm phi ngữ cảnh?

Việc sử dụng CFG cho JSON không chỉ là lý thuyết mà mang lại những lợi ích thực tế sau:

- Xử lý cấu trúc lồng nhau vô hạn: Giống như cách máy PDA có thể dùng ngăn xếp để kiểm tra các xâu nhập dài và phức tạp như aabaaacbbbabb, các trình phân tích JSON (JSON Parsers) sử dụng ngăn xếp để theo dõi các lớp ngoặc { hoặc [đang mở.

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

- Phân tích cú pháp (Parsing): Các quy tắc trên cho phép trình biên dịch xây dựng một Cây phân tích cú pháp (Parse Tree). Cây này giúp máy tính hiểu được "địa chỉ" của từng mẩu dữ liệu trong một tệp JSON khổng lồ.
- Kiểm tra tính hợp lệ: Nếu một tệp JSON thiếu một dấu ngoặc đóng hoặc thừa một dấu phẩy, trình phân tích cú pháp dựa trên CFG sẽ phát hiện ra ngay lập tức vì chuỗi đó không thể được sinh ra từ bộ quy tắc.

Bất cứ ai ngừng học tập đều già đi, dù ở tuổi 20 hay 80. Bất cứ ai tiếp tục học tập đều trẻ mãi!

HENRY FORD

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN